



**COMSATS University Islamabad,
Sahiwal Campus, Pakistan**

Grocify - The Online Supermart

**A project presented to
COMSATS Institute Islamabad, Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Software Engineering (2022-2026)

By

**Ghulam Farid
Ammara**

**CIIT/FA22-BSE-071/SWL
CIIT/ FA22-BSE-078/SWL**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Ghulam Farid

Ammara

Certificate of Approval

It is to certify that the final year project of BS (SE) “Project title” was developed by **Ghulam Farid (CIIT/FA22-BSE-071)** and **Ammara (CIIT/FA22-BSE-078)** under the supervision of “**KHURRAM NISSAR**” and co supervisor “**SHEHZAD ALI**” and that in (their/his/her) opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

Mr. Khurram Nissar

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Head of Department:

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Grocify is a comprehensive Android-based grocery delivery application developed for Pakistan's rapidly growing digital market. It addresses critical gaps in existing platforms like Box.com.pk, QnE, Instamart, and Foodpanda, which lack real-time tracking, multilingual search, AI personalization, and financial management tools.

The system serves three stakeholders Customers, Vendors, and Administrators built on a three-tier architecture using Kotlin (Android), Node.js/Express.js (backend), and MySQL (database).

Key features include real-time GPS delivery tracking, Urdu/English voice search, AI-driven product recommendations, post-order modification, wishlist management, recipe-based ordering, flash offers, and a Smart Budgeting module. Vendors get a dedicated dashboard for inventory, analytics, and order management, while Administrators have centralized system control.

Security is ensured through JWT authentication, bcrypt hashing, SSL/TLS encryption, and role-based access control. Push notifications are handled via Firebase Cloud Messaging (FCM).

Developed using Agile methodology with full SDLC documentation, the system passed 214 test assertions at a 100% pass rate across unit, integration, performance, and security testing.

Grocify is a scalable, market-ready platform with future expansion planned for deep learning recommendations, digital payments, computer vision ordering, and iOS support.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Mr. Khurram Nissar**” and our Co-Supervisor “**Mr. Shehzad Ali**”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Ghulam Farid

Ammara

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ER	Entity-Relationship
FCM	Firebase Cloud Messaging
FR	Functional Requirement
FYP	Final Year Project
GPS	Global Positioning System
GUI	Graphical User Interface
HTTP/HTTPS	HyperText Transfer Protocol / Secure
JWT	JSON Web Token
MySQL	My Structured Query Language
NFR	Non-Functional Requirement
NLP	Natural Language Processing
OOP	Object-Oriented Programming
PKR	Pakistani Rupee
REST	Representational State Transfer
RTM	Requirements Traceability Matrix
SDD	Software Design Description
SDLC	Software Development Life Cycle
SQL	Structured Query Language
SRS	Software Requirements Specification
SSL/TLS	Secure Sockets Layer / Transport Layer Security
UAT	User Acceptance Testing

UI/UX	User Interface / User Experience
UML	Unified Modeling Language

Table of Contents

1	Introduction.....	1
1.1	Brief.....	2
1.2	Relevance to Course Modules.....	3
1.2.1	Object-Oriented Programming (OOP):.....	3
1.2.2	Software Engineering:	3
1.2.3	Database Systems:	3
1.2.4	Mobile Application Development:	4
1.2.5	Computer Networks and Web Technologies:	4
1.2.6	Artificial Intelligence:.....	4
1.2.7	Human-Computer Interaction:	4
1.3	Project Background	4
1.4	Literature Review	5
1.5	Analysis from Literature Review	8
1.6	Methodology and Software Lifecycle	8
1.6.1	Selected Methodology and SDLC Model	8
1.6.2	Rationale behind Selected Methodology	9
2	Problem Definition.....	11
2.1	Problem Statement	11
2.2	Deliverables and Development Requirements	12
2.3	Current System Analysis.....	13
3	Requirement Analysis.....	16
3.1	Requirement Elicitation Approach.....	16
3.2	Use Case Diagrams	17
3.3	Use Case Descriptions.....	20
3.3.1	UC-1: Register / Login.....	20
3.3.2	UC-2: Update Profile	21
3.3.3	UC-3: Browse & Search Products	21
3.3.4	UC-4: View Product Details	22
3.3.5	UC-5: Add / Edit / Delete Products	22
3.3.6	UC-6: Add to Cart.....	23
3.3.7	UC-7: Review Cart	23
3.3.8	UC-8: Place Order.....	24
3.3.9	UC-9: View & Manage Orders	25
3.3.10	UC-10: Track Order (Real-Time)	25

3.3.11	UC-11: Modify / Cancel Order	26
3.3.12	UC-12: Accept / Dispatch Orders	26
3.3.13	UC-13: Wishlist Management	27
3.3.14	UC-14: Provide Feedback & Rating	27
3.3.15	UC-15: Add Flash Offers / Discounts.....	28
3.3.16	UC-16: View Reports & Earnings	28
3.3.17	UC-17: Add / Manage Users.....	29
3.4	Functional Requirements.....	29
3.4.1	FR-1 User Management & Authentication	30
3.4.2	FR-2 Product Browsing & Smart Search.....	30
3.4.3	FR-3 Shopping Cart & Order Placement	31
3.4.4	FR-4 Order Tracking & Management.....	31
3.4.5	FR-5 Product & Inventory Management	32
3.4.6	FR-6 Feedback & Rating System	32
3.4.7	FR-7 Smart Notifications & Promotions	33
3.4.8	FR-8 Budgeting & Expense Tracking.....	33
3.4.9	FR-9 Recipe & Product Integration	34
3.4.10	FR-10 Admin Dashboard & Control.....	34
3.5	Non-Functional Requirements	35
3.5.1	Usability Requirements.....	35
3.5.2	Performance Requirements.....	35
3.5.3	Security Requirements	36
3.5.4	Reliability Requirements	36
3.5.5	Scalability Requirements	36
3.5.6	Portability Requirements	37
4	Design and Architecture	38
4.1	System Architecture	38
4.1.1	Presentation Layer	38
4.1.2	Application Layer	39
4.1.3	Data Layer.....	39
4.2	Data Representation	40
4.2.1	Entity-Relationship Model.....	40
4.2.2	Data Dictionary	40
4.3	Process Flow Representation	45
4.3.1	Product Browsing and Smart Search Flow	45
4.3.2	Order Placement Flow	47
4.3.3	Real-Time Order Tracking Flow	49

4.3.4	Vendor Inventory Update Flow	51
4.3.5	Admin Dispatching Flow	53
4.4	Design Models.....	55
4.4.1	Class Diagram.....	55
4.4.2	Sequence Diagrams.....	57
4.4.3	State Transition Diagram — Order Lifecycle.....	60
4.4.4	Data Flow Diagrams	61
4.5	Requirements Traceability Matrix	64
5	Implementation	66
5.1	Development Environment and Technology Stack.....	66
5.2	Algorithms.....	67
5.2.1	User Authentication Algorithm.....	67
5.2.2	Smart Product Search Algorithm	68
5.2.3	Order Placement Algorithm	68
5.2.4	Real-Time Order Tracking Algorithm	69
5.2.5	Vendor Inventory Update Algorithm.....	69
5.2.6	Budget Tracking and Alert Algorithm	70
5.3	External API Integrations.....	70
5.4	User Interface Design.....	71
5.4.1	Login Screen	72
5.4.2	User Profile	73
5.4.3	Home Screen.....	74
5.4.4	Search Screen.....	75
5.4.5	Categories	76
5.4.6	Review Cart Screen.....	77
5.4.7	Order Confirmation Screen.....	78
5.4.8	Admin Dashboard	79
6	Testing and Evaluation	80
6.1	Testing Methodology	80
6.2	Unit Testing.....	81
6.2.1	User Authentication Module.....	81
6.2.2	Product Search Module.....	82
6.2.3	Order Placement Module	83
6.3	Functional Testing.....	85
6.3.1	Multi-Role Authentication	85
6.3.2	Order Lifecycle Management	86
6.3.3	Vendor Inventory Management	87

6.4	Integration Testing	88
6.5	Non-Functional Requirements Testing	90
6.5.1	Performance Testing	90
6.5.2	Security Testing	91
6.6	Automated Testing	92
7	Conclusion and Future Work	94
7.1	Conclusion.....	94
7.2	Limitations of the Current Implementation.....	95
7.3	Future Work	96
8	References.....	98

List of Figures

Figure 3.1: Admin Side.....	18
Figure 3.2: User Side	19
Figure 4.1: Product Browsing & Smart Search	46
Figure 4.2: Order Placement.....	48
Figure 4.3: Order Tracking	50
Figure 4.4: Vendor Inventory Update.....	52
Figure 4.5: Admin Dispatching.....	54
Figure 4.6: Class Diagram	56
Figure 4.7: Place Order.....	58
Figure 4.8: Vendor Updating Inventory.....	59
Figure 4.9: Admin Dispatching.....	60
Figure 4.10: Order Lifecycle.....	61
Figure 4.11: DFD – Level 0.....	62
Figure 4.12: DFD – Level 1 (A)	62
Figure 4.13: DFD – Level 1(B).....	63
Figure 4.14: DFD – Level 1(C).....	63
Figure 4.15: DFD – Level 2.....	64
Figure 4.16: Traceability Matrix.....	65
Figure 5.1: Login Screen.....	72
Figure 5.2: User Profile.....	73
Figure 5.3: Home Screen	74
Figure 5.4: Search Screen	75
Figure 5.5: Categories.....	76
Figure 5.6: Review Cart Screen.....	77
Figure 5.7: Order Confirmation Screen	78
Figure 5.8: Admin Dashboard.....	79

List of Tables

Table 1.1: Comparative Analysis of Existing Systems and Proposed Solutions	6
Table 2.1: Comparative Analysis of Existing Grocery Platforms vs. Grocify	14
Table 3.1: UC-1: Register / Login	20
Table 3.2: UC-2: Update Profile	21
Table 3.3: UC-3: Browse & Search Products	21
Table 3.4: UC-4: View Product Details	22
Table 3.5: UC-5: Add / Edit / Delete Products	22
Table 3.6: UC-6 Add to Cart.....	23
Table 3.7: UC-7 Review Cart	24
Table 3.8: UC-8: Place Order	24
Table 3.9: UC-9: View & Manage Orders.....	25
Table 3.10: UC-10: Track Order (Real-Time).....	25
Table 3.11: UC-11: Modify / Cancel Order.....	26
Table 3.12: UC-12: Accept / Dispatch Orders.....	26
Table 3.13: UC-13: Wishlist Management	27
Table 3.14: UC-14: Provide Feedback & Rating.....	27
Table 3.15: UC-15: Add Flash Offers / Discounts	28
Table 3.16: UC-16: View Reports & Earnings.....	28
Table 3.17: UC-17: Add / Manage Users	29
Table 3.18: FR-1 User Management & Authentication.....	30
Table 3.19: FR-2 Product Browsing & Smart Search.....	30
Table 3.20: FR-3 Shopping Cart & Order Placement.....	31
Table 3.21: FR-4 Order Tracking & Management	31
Table 3.22: FR-5 Product & Inventory Management	32
Table 3.23: FR-6 Feedback & Rating System	32
Table 3.24: FR-7 Smart Notifications & Promotions	33
Table 3.25: FR-8 Budgeting & Expense Tracking	33
Table 3.26: FR-9 Recipe & Product Integration.....	34
Table 3.27: FR-10 Admin Dashboard & Control	35
Table 4.1: User Table.....	41
Table 4.2: Order Table.....	42

Table 4.3: Product Table.....	43
Table 4.4: Inventory Table.....	43
Table 4.5: Payment Table	44
Table 5.1: Grocify Technology Stack.....	67
Table 5.2: Grocify Technology Stack.....	70
Table 6.1: User Authentication	81
Table 6.2: Product Search	83
Table 6.3: Order Placement	84
Table 6.4: Role-Based Access	85
Table 6.5: Order Lifecycle Management	86
Table 6.6: Vendor Inventory Management.....	88
Table 6.7: Integration Test Cases.....	89
Table 6.8: Performance Test Results	91
Table 6.9: Security Test Results	92
Table 6.10: Automated Testing Summary	93

CHAPTER 1

INTRODUCTION

1 INTRODUCTION

Grocify – The Online Supermart is a mobile-based intelligent grocery delivery system developed as a Final Year Project at COMSATS University Islamabad, Sahiwal Campus. The system targets a clearly identified gap in Pakistan's mobile grocery market by building a fully integrated, AI-assisted platform that connects customers, vendors, and administrators within a single digital ecosystem.

Pakistan's e-commerce sector has grown consistently over recent years, driven by rising smartphone penetration, expanding mobile internet access, and an increasingly urban population of over 240 million. The grocery delivery segment remains one of the least developed areas in this space. Existing platforms such as Box.com.pk, QnE, and Instamart offer only basic catalogs and ordering interfaces. Customers have no real-time order visibility, no native-language search, no way to correct orders before dispatch, and no spending management tools. Vendors lack the inventory and reporting infrastructure they need to operate efficiently. Grocify was built specifically to address these gaps.

Rather than iterating on an existing solution, Grocify was designed from scratch for the Pakistani market. The application includes GPS-based real-time order tracking, bilingual voice search in Urdu and English, AI-driven product recommendations, post-order modification before dispatch, wishlist management, recipe-based ingredient ordering, flash offers, and monthly budget tracking. Vendors get a full dashboard covering inventory management, order processing, customer feedback, and sales reporting. A separate Administrator Panel handles platform-wide oversight.

The system is built on a three-tier architecture: a Kotlin Android client at the presentation layer, a Node.js and Express.js REST API at the application layer, and a normalized MySQL database at the data layer. Security is implemented through JWT authentication, bcrypt password hashing, SSL/TLS encryption, and role-based access control. The Google Maps API, Google Speech-to-Text API, and Firebase Cloud Messaging handle real-time, location-aware, and notification features. Development followed the Agile methodology across six sprint-based modules.

Academically, Grocify applies knowledge from Object-Oriented Programming, Software Engineering, Database Systems, Mobile Application Development, Computer Networks, Human-Computer Interaction, and Artificial Intelligence. The full Software Development Life Cycle was followed, producing a Scope Document, Software Requirements Specification, Software Design Description, and this Final Report. The system also has realistic potential as a commercially deployable platform in Pakistan's growing digital grocery market.

1.1 Brief

Grocify – The Online Supermart is a smart, AI-assisted mobile grocery delivery application developed for the Android platform. The system is designed to address the critical limitations present in existing grocery delivery platforms operating in Pakistan's digital market, including the absence of real-time order tracking, lack of multilingual product search, no post-order modification capability, and inadequate vendor management tools.

The application connects three primary stakeholders Customers, Vendors, and Administrators within a single unified platform. Customers can browse and search products, place and track orders in real time, manage a wishlist, set monthly grocery budgets, and receive AI-driven personalized recommendations. Vendors are equipped with a comprehensive dashboard for inventory management, order processing, and sales analytics. Administrators maintain centralized control over all platform operations through a dedicated Admin Panel.

The system is built using Kotlin for the Android frontend, Node.js with Express.js for the RESTful backend API, and MySQL as the relational database. External integrations include the Google Maps API for real-time delivery tracking, the Google Speech-to-Text API for bilingual voice search in Urdu and English, and Firebase Cloud Messaging (FCM) for real-time push notifications. Security is enforced through JWT-based authentication, bcrypt password hashing, and SSL/TLS-encrypted communication.

The project was developed following the Agile Software Development Methodology with an Object-Oriented Programming design approach, organized across six sprint-based functional modules. The complete Software Development Life Cycle was followed, producing a Scope Document, Software Requirements Specification (SRS), Software

Design Description (SDD), and this Final Report as formal project artifacts. The remainder of this report is organized as follows: Chapter 2 defines the problem, Chapter 3 presents the requirements analysis, Chapter 4 describes the system design and architecture, Chapter 5 discusses implementation details, Chapter 6 presents testing and evaluation results, and Chapter 7 concludes with future work directions.

1.2 Relevance to Course Modules

The development of Grocify directly applied knowledge acquired across multiple core courses of the Bachelor of Science in Software Engineering program at COMSATS University Islamabad. Each major component of the system draws on one or more of the following courses.

1.2.1 Object-Oriented Programming (OOP):

The entire system design is based on OOP principles. System entities such as User, Vendor, Product, Order, Cart, Payment, and Inventory are modeled as classes with well-defined attributes, methods, and relationships. Concepts of encapsulation, inheritance, and polymorphism are applied throughout the Kotlin Android codebase and the Node.js service layer.

1.2.2 Software Engineering:

The project followed the complete Software Development Life Cycle, from requirements elicitation and use case modeling through architectural design, implementation, testing, and documentation. The SRS and SDD artifacts produced for this project directly reflect the documentation standards and modeling techniques taught in this course, including UML diagrams such as class diagrams, sequence diagrams, state transition diagrams, and data flow diagrams.

1.2.3 Database Systems:

The MySQL database schema was designed by applying relational database principles including normalization to Third Normal Form (3NF), definition of primary and foreign key relationships, enforcement of referential integrity through cascade rules, and

optimization through indexing on frequently queried columns such as Product.name, Product.category, and Order.status.

1.2.4 Mobile Application Development:

The Android application was developed in Kotlin using Android Studio, applying concepts of Activities, Fragments, ViewModels following the MVVM architectural pattern, RecyclerView adapters for dynamic list rendering, and the Android Jetpack Navigation Component for screen-to-screen navigation management.

1.2.5 Computer Networks and Web Technologies:

The RESTful API was designed applying HTTP protocol semantics, REST architectural constraints, JSON data interchange, and TLS-based encryption. WebSocket protocol knowledge was applied in the real-time order tracking update mechanism. JWT token-based authentication reflects understanding of stateless session management over HTTP.

1.2.6 Artificial Intelligence:

The AI-driven product recommendation engine applies collaborative filtering principles, analyzing user purchase history and browsing behavior to generate personalized product suggestions. The integration of the Google Speech-to-Text API applies concepts of natural language processing and speech recognition.

1.2.7 Human-Computer Interaction:

The application's user interface was designed following Google's Material Design 3 guidelines, applying principles of visual hierarchy, consistency, accessibility, and minimizing cognitive load for users with varying levels of digital literacy.

1.3 Project Background

Pakistan's grocery retail sector has traditionally been dominated by physical stores, open-air markets, and informal vendor networks. However, the rapid proliferation of affordable Android smartphones, the expansion of 4G mobile internet infrastructure, and the growing familiarity with digital payment platforms such as JazzCash and Easypaisa have

collectively created conditions highly favorable to the growth of mobile-based grocery services.

The idea behind Grocify emerged from a direct observation of the friction points experienced by grocery shoppers in the Sahiwal region. Consumers expressed consistent frustrations with the experience of shopping for groceries, both physically and digitally. In physical stores, the time cost of travel, the inability to compare prices across vendors, and the challenge of managing grocery budgets across multiple visits represent significant pain points. In existing digital platforms, the limitations are equally significant orders are placed with no certainty about when or how they will arrive, product discovery requires knowing the exact English name of an item, and mistakes in an order cannot be corrected after placement.

Grocify is an online grocery delivery system a digital platform that connects consumers with grocery vendors through a mobile application, enabling them to browse a product catalog, add items to a cart, place orders, make payments, and receive deliveries at their door. The concept extends beyond simple ordering by incorporating intelligent features: the system learns from a user's purchase history to make personalized suggestions, it listens to voice commands in Urdu and English to assist product discovery, and it maintains a live map of the delivery rider's location so the customer always knows exactly where their order is. For vendors, the system functions as a complete digital storefront and operations management tool, replacing manual inventory tracking and informal order communication with structured, real-time digital workflows.

The motivation for building Grocify specifically for the Pakistani market reflects the recognition that international grocery platforms are not well-suited to local needs. Language barriers, the dominance of cash-on-delivery as a payment preference, the informal structure of many small grocery vendors, and the specific product categories relevant to Pakistani households all require a locally designed and locally adapted solution.

1.4 Literature Review

This section presents a review of current grocery delivery platforms, related products, and existing systems relevant to the Grocify project. The review examines the functional capabilities and documented weaknesses of platforms currently operating in the Pakistani

and regional grocery delivery market, and identifies the specific gaps that the proposed system is designed to address.

A systematic examination of four prominent grocery delivery platforms Box Online Supermarket, Grocers App, Instamart, and QnE (Quick and Easy) was conducted to identify recurring functional limitations and opportunities for improvement. Each platform was evaluated against a set of criteria derived from user research and academic literature on mobile commerce best practices, including real-time order transparency, personalization capability, order management flexibility, user engagement features, and delivery service options. The findings of this comparative analysis are summarized in Table 1.1 below.

Table 1.1: Comparative Analysis of Existing Systems and Proposed Solutions

Application Name	Weakness	Proposed Project Solution
Box – Online Supermarket	Lacks real-time order tracking and offers only basic, non-personalized product suggestions.	Grocify includes real-time GPS-based order tracking for delivery transparency and an AI-powered recommendation engine for personalization. Users receive continuous delivery status updates and product suggestions tailored to their individual purchase behavior and browsing history. These enhancements significantly improve usability and overall customer satisfaction.
Grocers App	Does not support post-order customization or editing of placed orders after confirmation.	Grocify allows users to modify item quantities or cancel orders entirely at any point before the order is dispatched. This flexibility reduces order-related errors and increases user convenience. It directly addresses one of the most significant usability limitations identified in existing grocery platforms.

Instamart	Missing Wishlist feature, which complicates repeat purchases and reduces overall user convenience.	Grocify includes a dedicated Wishlist Management module where users can save and manage their favorite or frequently purchased items for future reference. This feature accelerates the shopping process for recurring grocery needs and enhances the personalized experience for returning users.
QnE (Quick and Easy)	Lacks AI-driven personalization and modern delivery options such as slot bidding and emergency express delivery. User engagement features including smart notifications, recipe integration, and chatbot support are also entirely absent.	Grocify provides AI-driven product recommendations, delivery slot bidding, an emergency express delivery mode, and flash offer management. The system additionally incorporates smart context-aware push notifications, recipe-based ingredient ordering, and an AI-powered chatbot for real-time customer assistance, collectively making the shopping experience significantly more engaging and efficient.

The comparative analysis reveals significant gaps across existing Pakistani grocery delivery platforms. Real-time tracking is absent from most Box Online Supermarket relies entirely on static status updates, the most cited customer frustration. Order management is equally rigid, treating confirmed orders as immutable despite grocery requirements frequently changing. Personalization is superficial or missing entirely, with neither Box nor QnE offering behavioral recommendations, and Instamart lacking even a Wishlist feature. Beyond these core gaps, platforms like QnE offer no recipe integration, chatbot support, flash offers, or flexible delivery options, reflecting a broader failure to move beyond basic transactional functionality. Grocify directly addresses each of these limitations within a single, cohesive platform something no existing competitor in the Pakistani market currently offers.

1.5 Analysis from Literature Review

The literature review confirms strong alignment between academic research and Grocify's core design decisions. Studies on mobile commerce consistently identify post-purchase uncertainty as the primary consumer dissatisfier, yet no existing Pakistani grocery platform offers meaningful real-time tracking. Grocify addresses this through GPS-based live delivery tracking via the Google Maps API, supplemented by automated FCM notifications at each order stage. The work of He et al. on Neural Collaborative Filtering and Ning and Karypis on sparse linear methods establishes that behavioral personalization consistently outperforms popularity-based recommendations in engagement and conversion a capability entirely absent from existing local platforms that Grocify directly captures through its AI-driven recommendation engine. Hameed et al.'s finding that language barriers drive mobile commerce non-adoption among non-English users in Pakistan provides direct justification for Grocify's bilingual voice search, implemented via Google Speech-to-Text with support for both Urdu (ur-PK) and English (en-PK) models. Finally, Grocify's security architecture JWT authentication, bcrypt hashing, TLS 1.3, and role-based access control meets the standards set by Rescorla's TLS 1.3 RFC and IETF JWT guidelines. Overall, the review confirms that Grocify's feature set is evidence-based, addresses documented gaps in the local market, and follows best practices in mobile commerce system development.

1.6 Methodology and Software Lifecycle

This section discusses the software development methodology and the Software Development Life Cycle (SDLC) model selected for the Grocify – The Online Supermart project. It explains how the development process was structured, how tasks were organized and executed across the project timeline, and why the selected approach was most appropriate for the nature and requirements of this project.

1.6.1 Selected Methodology and SDLC Model

The Grocify project was developed following the Agile Software Development Methodology combined with the Iterative and Incremental Software Development Life Cycle. The project was divided into six structured sprints, each corresponding to one of the six functional modules defined in the project scope: User Management and Authentication, Product Browsing and Smart Search, Order Management and Delivery, Personalization and

Engagement, Financial Management and Expense Tracking, and Vendor and Inventory Management.

Each sprint followed a consistent cycle of planning, design, implementation, testing, and review. At the end of each sprint, the completed module was demonstrated to the project supervisor for feedback, and any required modifications were incorporated before proceeding to the next sprint. Integration of all modules was performed incrementally, with inter-module communication tested at each integration milestone. The final two phases of the project system-wide integration testing and documentation were conducted after all six modules were completed and individually validated.

The Object-Oriented Programming (OOP) design methodology was applied throughout the system design phase. The system's entities were modeled as classes with well-defined attributes, methods, and inter-class relationships, documented through UML class diagrams. The MVVM architectural pattern was applied within the Android application layer to separate UI logic from business logic and data access concerns.

1.6.2 Rationale behind Selected Methodology

The Agile methodology was selected over alternative models such as the Waterfall model for several specific reasons directly related to the characteristics of the Grocify project.

Accommodation of evolving requirements: Features such as the AI recommendation engine, voice search accuracy parameters, and the GPS tracking update frequency required iterative refinement based on testing outcomes. The Agile model's acceptance of changing requirements between sprints allowed these features to be refined progressively without disrupting the overall project structure.

Alignment with academic supervision: The sprint-based structure provided natural checkpoints for supervisor review and feedback. Regular demonstrations of completed functionality allowed Mr. Khurram Nissar to provide timely, specific guidance that was incorporated into subsequent sprints, ensuring close alignment between the project's direction and academic expectations.

Risk reduction through incremental delivery: By delivering and testing each module independently before integration, the team was able to identify and resolve defects at the

module level rather than encountering them during late-stage integration, significantly reducing the risk of schedule-threatening bugs.

Suitability for a two-person team: The Agile methodology's emphasis on lightweight documentation during active development, combined with clearly defined sprint tasks, provided an effective framework for dividing work between the two team members Ghulam Farid responsible for backend development and Ammara responsible for frontend and UI/UX while maintaining overall project coherence.

The Object-Oriented methodology was selected because the system's domain naturally maps to a collection of interacting entities Users, Products, Orders, Vendors, Carts, and Payments each with well-defined state and behavior. OOP's principles of encapsulation, inheritance, and polymorphism provided a principled basis for designing a modular, maintainable, and extensible codebase that can accommodate the future enhancements identified in the project's roadmap.

CHAPTER 2

PROBLEM DEFINITION

2 PROBLEM DEFINITION

This chapter provides a structured and detailed definition of the problem that Grocify – The Online Supermart is designed to address. It presents the formal problem statement, the deliverables and development requirements of the project, and an analysis of the current system landscape against which Grocify is positioned.

2.1 Problem Statement

The rapid expansion of mobile commerce in Pakistan has created significant consumer demand for digital grocery services. However, the current generation of grocery delivery platforms operating in this market is characterized by a systemic inadequacy that spans multiple dimensions of the user experience, vendor operations, and system functionality. A rigorous analysis of these platforms, complemented by primary user research, reveals a consistent pattern of functional deficiency, accessibility limitation, and operational inefficiency that constitutes the core problem this project seeks to resolve.

At the consumer level, the most critical deficiency is the absence of real-time order transparency. Studies on consumer behavior in online retail consistently demonstrate that post-purchase anxiety uncertainty about whether an order has been processed, accepted, and dispatched is one of the primary drivers of customer dissatisfaction and platform abandonment [3]. On currently available Pakistani grocery platforms, once a customer places an order, the system typically provides nothing more than a static confirmation message. There is no live tracking, no automated status progression updates, and no estimated delivery time. Customers are left to wait indefinitely with no mechanism for gauging order progress, creating a fundamentally unsatisfactory experience.

Product discovery on existing platforms is equally problematic. The product search interfaces on platforms such as Box.com.pk and QnE rely on simple lexical matching that fails to handle natural language queries, near-matches, or Urdu-language input. For a population where a significant proportion is more comfortable conducting daily activities in Urdu than in English, this represents a fundamental accessibility barrier. A study of mobile application accessibility in South Asian markets highlights that language exclusivity in digital platforms directly correlates with lower adoption rates among non-English-proficient demographics [4]. The absence of voice input compounds this problem, as voice-based interaction is increasingly the preferred modality for mobile users globally.

Order management inflexibility is a further source of significant user frustration. In their current form, grocery delivery platforms in Pakistan treat an order as immutable upon confirmation. This design assumption is fundamentally at odds with the reality of grocery shopping, where items are frequently added, removed, or substituted as users remember additional requirements or change their minds. The inability to modify orders before dispatch routinely results in incorrect deliveries, wasted food, unnecessary expense, and disputes that strain the vendor-customer relationship.

From the vendor perspective, existing platforms provide inadequate tools for effective inventory and order management. Vendors typically lack real-time visibility into their stock levels as orders are placed, receive order notifications through manual processes that are prone to error and delay, and have no access to structured analytics about their sales performance, popular products, or customer satisfaction levels. This leaves vendors operationally reactive rather than proactively managed, limiting their ability to optimize their businesses.

The absence of personalization features represents a missed opportunity of significant magnitude. Research in consumer behavior and recommendation systems consistently demonstrates that personalized product recommendations drive substantially higher conversion rates, larger basket sizes, and stronger customer retention than generic product displays [7]. The complete absence of any form of behavioral personalization on existing Pakistani grocery platforms means that users who have demonstrated clear purchasing preferences receive no benefit from their order history.

Finally, the absence of financial management tools represents a gap in customer value delivery. For many Pakistani households managing tight monthly budgets, the ability to set a grocery spending limit, track cumulative expenditure across the month, and receive alerts before overspending would constitute a genuinely valuable service that differentiates a platform from its competitors.

2.2 Deliverables and Development Requirements

The following deliverables were defined and completed as part of the Grocify project.

The **primary software deliverable** is a fully functional Android mobile application (APK file) supporting three user roles Customer, Vendor, and Administrator each with a

dedicated interface and role-appropriate feature set. The application is compatible with Android 8.0 (Oreo) and above.

The **backend deliverable** is a RESTful API built using Node.js version 18.x with the Express.js framework, providing forty-seven API endpoints covering all system functions. The API is documented and follows REST architectural conventions including resource-based URL design, appropriate HTTP method usage (GET, POST, PUT, DELETE), and standard HTTP response code semantics.

The **data layer deliverable** is a fully normalized MySQL relational database schema comprising sixteen tables with well-defined foreign key relationships, appropriate indexing on frequently queried fields, and referential integrity constraints to maintain data consistency.

Integration deliverables include a functioning Google Maps API integration for real-time delivery tracking, a Google Speech-to-Text API integration for voice-based search, a Firebase Cloud Messaging integration for push notifications, and JWT-based authentication middleware for secure session management.

Academic documentation deliverables, all completed prior to this Final Report, include the Scope Document (Project Proposal), Software Requirements Specification (SRS), and Software Design Description (SDD).

Development requirements included the Android Studio Flamingo Integrated Development Environment, Kotlin 1.9 programming language, Node.js version 18.x runtime, MySQL version 8.0, XAMPP for local development environment setup, Git and GitHub for version control and collaboration, Figma for UI/UX wireframe design, and Postman for API development and testing.

2.3 Current System Analysis

There is no single, comprehensive, purpose-built grocery delivery platform currently serving the local Sahiwal market. At the national level, several platforms offer partial solutions, each with documented limitations.

Box – The Online Supermarket (box.com.pk) is among Pakistan's more established online grocery platforms and offers a reasonable product catalog. However, it lacks real-time order

tracking, providing only static status updates. Its product recommendation engine is basic, offering generic category suggestions rather than behavior-driven personalization. Post-order modification is not supported, and there is no budgeting or financial tracking functionality.

QnE (Quick and Easy) presents a similar set of limitations. The platform's search functionality is restricted to English keyword matching, and there is no voice search capability. AI-driven personalization, delivery slot management, recipe integration, and chatbot assistance are entirely absent. The vendor management tools available on this platform are minimal and do not support meaningful analytics.

Instamart, operating primarily in larger metropolitan areas, offers delivery speed as its primary differentiator but lacks wishlist functionality, post-order modification, financial management tools, and meaningful personalization features.

Foodpanda's grocery module benefits from the platform's existing infrastructure but focuses primarily on restaurant food delivery with grocery as a secondary offering. The grocery interface lacks the depth of features including vendor-specific inventory management, budgeting tools, and AI recommendation quality needed for a full-featured online supermart experience.

The comparative analysis presented in Table 2.3.1 summarizes the feature gaps across these existing platforms relative to Grocify's feature set.

Table 2.1: Comparative Analysis of Existing Grocery Platforms vs. Grocify

Feature	Box	QnE	Instamart	Foodpanda Grocery	Grocify (Proposed)
Real-Time GPS Tracking	No	No	Partial	Partial	Yes
Urdu Voice Search	No	No	No	No	Yes
AI Product Recommendations	Basic	No	No	Partial	Yes
Post-Order Modification	No	No	No	No	Yes
Wishlist Management	No	No	No	No	Yes

Vendor Analytics Dashboard	No	No	No	Partial	Yes
Budget & Expense Tracking	No	No	No	No	Yes
Recipe Integration	No	No	No	No	Yes
Flash Offer Management	Partial	No	Partial	Yes	Yes
JWT Security + HTTPS	Unknown	Unknown	Unknown	Yes	Yes

This analysis confirms that no existing platform currently offers the comprehensive, integrated feature set that Grocify provides. Grocify is developed not as an incremental improvement to an existing system but as a novel platform built from the ground up to address the full spectrum of identified market needs.

CHAPTER 3

REQUIREMENT ANALYSIS

3 REQUIREMENT ANALYSIS

This chapter presents the complete requirements analysis for Grocify – The Online Supermart. Requirements were elicited through a combination of structured user interviews with potential customers and local grocery vendors, competitive analysis of existing grocery delivery platforms, and review of relevant academic literature on mobile commerce systems. The Use Case technique was adopted as the primary requirement modeling method, as it effectively represents the interactions between the system and its multiple stakeholder groups. This chapter includes use case diagrams, detailed use case descriptions, and a comprehensive specification of functional and non-functional requirements.

3.1 Requirement Elicitation Approach

A multi-source data gathering approach was employed to ensure that the requirements captured for Grocify reflect genuine user needs rather than assumptions. The approach comprised three primary methods.

First, semi-structured interviews were conducted with a sample of twelve potential end users in Sahiwal, including four university students, four working professionals, and four homemakers. These interviews explored participants' current grocery shopping habits, their experiences with and perceptions of existing digital grocery platforms, and their specific expectations for a new platform. Interview findings were analyzed using thematic analysis, yielding a set of recurring user needs and frustrations that directly informed the functional requirements.

Second, interviews were conducted with five local grocery vendors to understand the operational challenges they face in managing inventory, processing orders, and communicating with customers. These interviews highlighted the need for real-time inventory management tools, order notification systems, and customer feedback mechanisms.

Third, a systematic competitive analysis was performed on five existing grocery delivery platforms operating in Pakistan, examining their feature sets, user interface design, and documented user reviews. This analysis identified specific functional gaps and design weaknesses that Grocify's requirements were designed to address.

3.2 Use Case Diagrams

The Grocify system involves three primary actors: Customer, Vendor, and Administrator. Two use case diagrams were developed one representing the administrative operations managed by the Administrator actor, and one representing the customer-facing operations available to the Customer actor. Vendor use cases are partially represented within both diagrams where vendor interactions overlap with customer and administrative functions.

The **Admin Side Use Case Diagram** illustrates that the Administrator can log into the system, manage users and vendors, view and manage all orders in the system, add or modify product listings, accept and dispatch orders, add flash offers and discount campaigns, and view reports and earnings. The dependency relationship between 'Accept / Dispatch Orders' and 'View & Manage Orders' indicates that order dispatching necessarily requires prior order viewing. The 'include' relationship from 'Accept / Dispatch Orders' to 'Admin Login' indicates that authentication is a prerequisite for all administrative operations.

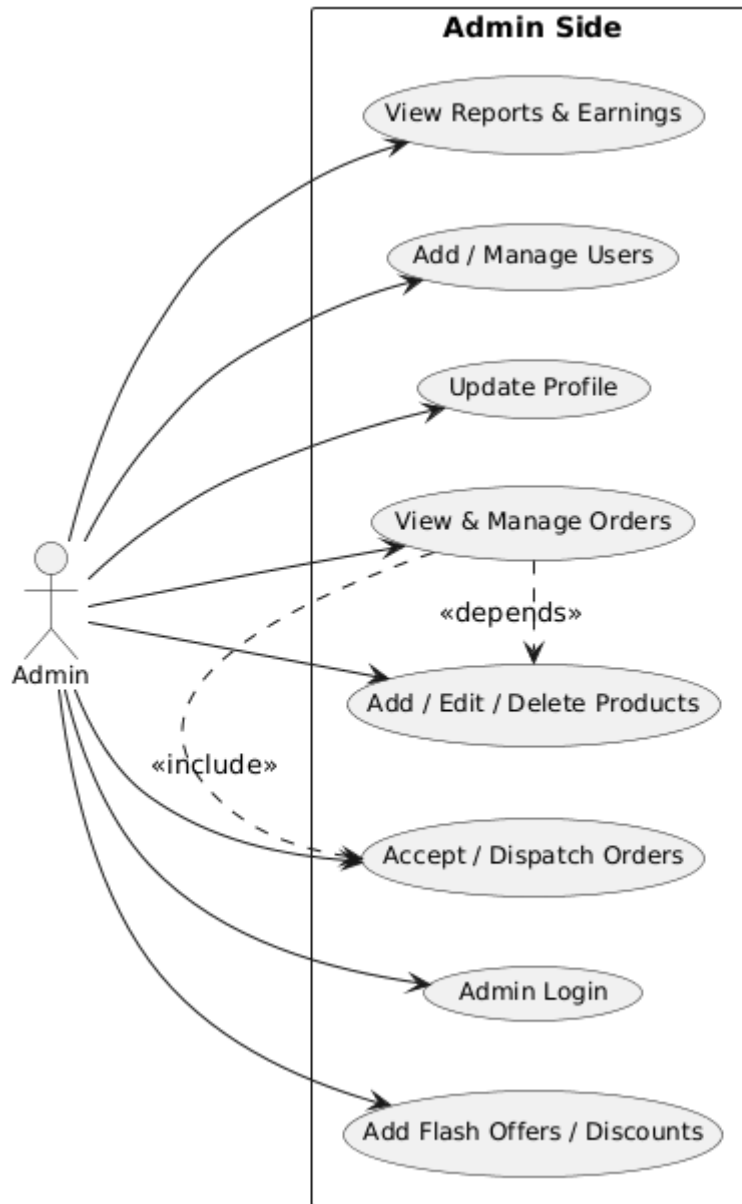


Figure 3.1: Admin Side

The **User Side Use Case Diagram** shows that a customer can register or log into the system, browse product categories, search and filter products, view detailed product information, add items to the shopping cart, review and modify the cart, place orders, track orders in real time, modify or cancel orders before dispatch, manage a personal wishlist, and provide feedback and ratings for completed orders. The 'include' relationships between 'Place Order,' 'Add to Cart,' and 'Review Cart' model the mandatory sequential dependency of these actions within the order placement workflow.

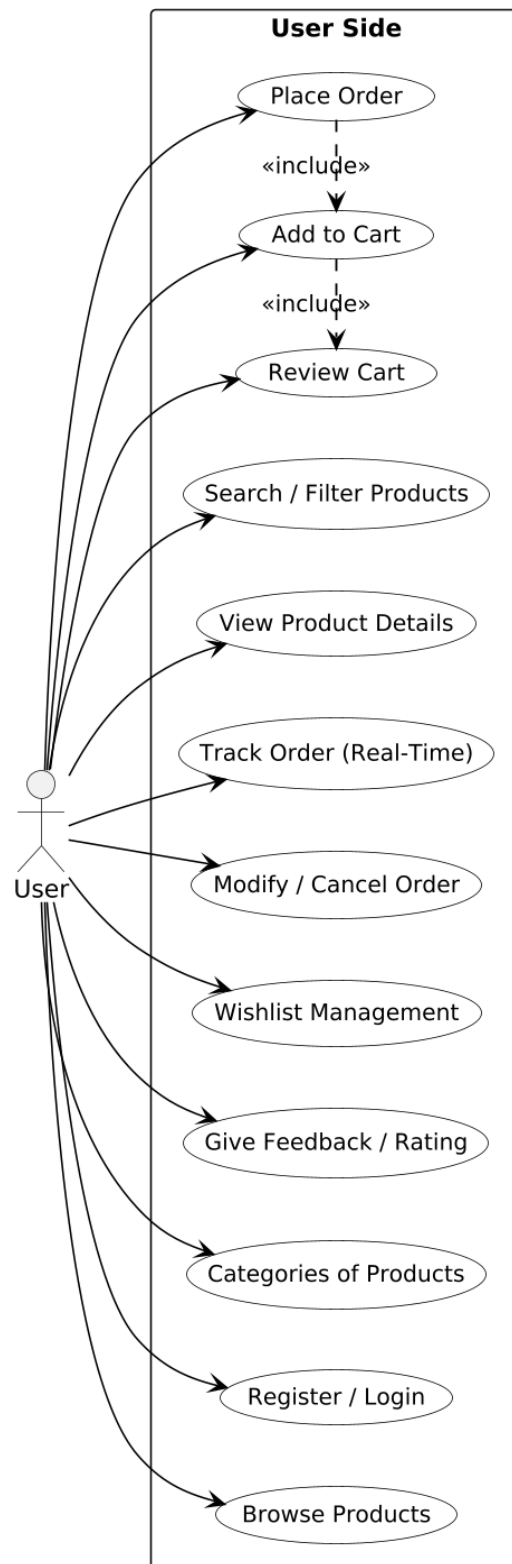


Figure 3.2: User Side

3.3 Use Case Descriptions

The use case diagram of Grocify – The Online Supermart illustrates the interaction between the system and its three primary actors: Customer, Vendor, and Admin. The Customer can register or log in, manage their profile, browse and search products, add items to the cart, place and track orders in real time, modify or cancel pending orders, manage wishlists, and provide feedback or ratings. The Vendor handles product listings by adding, editing, or deleting products, managing and dispatching customer orders, and applying discounts or flash offers. The admin supervises the overall system by managing users, monitoring orders, viewing reports and earnings, and maintaining platform-wide data accuracy and security. Overall, the diagram presents a clear overview of how each actor interacts with Grocify’s system to support a smooth, efficient, and user-friendly online grocery experience.

3.3.1 UC-1: Register / Login

This use case allows both Admin and User to register or log in to access their respective dashboards securely.

Table 3.1: UC-1: Register / Login

Use Case ID	UC-1
Use Case Name	Register / Login
Primary Actor	User, Admin
Description	Enables both User and Admin to register or log in to the system using valid credentials.
Trigger	User or Admin clicks on “Login” or “Register.”
Preconditions	Actor must have system access.
Postconditions	Actor is authenticated and redirected to their dashboard.
Normal Flow	1. Actor enters login/registration details. 2. System validates credentials. 3. Actor gains access.
Alternative Flows	Invalid credentials → show error.
Exceptions	Server downtime → access temporarily unavailable.
Business Rules	Password must meet complexity requirements.

Assumptions	User provides correct registration details.
-------------	---

3.3.2 UC-2: Update Profile

This use case lets both Admin and User update their personal information like name, email, and password.

Table 3.2: UC-2: Update Profile

Use Case ID	UC-2
Use Case Name	Update Profile
Primary Actor	User, Admin
Description	Allows modification of account details and credentials.
Trigger	User selects “Edit Profile.”
Preconditions	Actor must be logged in.
Postconditions	Updated data stored in database.
Normal Flow	1. Actor opens profile settings. 2. Updates details. 3. System saves changes.
Alternative Flows	Invalid data → show error message.
Exceptions	Network error → update fails.
Business Rules	Email must be unique in system.
Assumptions	User provides valid and updated information.

3.3.3 UC-3: Browse & Search Products

This use case allows users to explore, filter, and search for products using various criteria.

Table 3.3: UC-3: Browse & Search Products

Use Case ID	UC-3
Use Case Name	Browse & Search Products
Primary Actor	User
Description	Enables browsing or searching available products.
Trigger	User clicks “Browse” or enters search keywords.
Preconditions	Products must exist in catalog.
Postconditions	Filtered results displayed.

Normal Flow	1. User browses/searches. 2. System shows relevant results.
Alternative Flows	No matches → “No products found.”
Exceptions	Database connection lost.
Business Rules	Search results sorted by relevance.
Assumptions	Product catalog is updated regularly.

3.3.4 UC-4: View Product Details

This use case allows users to see full details of a selected product including price, stock, and description.

Table 3.4: UC-4: View Product Details

Use Case ID	UC-4
Use Case Name	View Product Details
Primary Actor	User
Description	Displays complete product info when clicked.
Trigger	User selects a product.
Preconditions	Product exists in database.
Postconditions	Product details displayed.
Normal Flow	1. User clicks product. 2. System loads details.
Alternative Flows	Product unavailable → show message.
Exceptions	Data fetch error.
Business Rules	Product details updated daily.
Assumptions	Product images and info are accurate.

3.3.5 UC-5: Add / Edit / Delete Products

This use case enables Admin to manage the product catalog by adding, updating, or removing products.

Table 3.5: UC-5: Add / Edit / Delete Products

Use Case ID	UC-5
Use Case Name	Add / Edit / Delete Products

Primary Actor	Admin
Description	Admin maintains the product catalog.
Trigger	Admin selects product management.
Preconditions	Admin must be logged in.
Postconditions	Product catalog updated.
Normal Flow	1. Admin adds/edits/deletes products. 2. System saves updates.
Alternative Flows	Invalid data → show validation error.
Exceptions	Database update fails.
Business Rules	Product name must be unique.
Assumptions	Admin enters accurate product data.

3.3.6 UC-6: Add to Cart

This use case allows users to add selected products to their shopping cart for purchase.

Table 3.6: UC-6 Add to Cart

Use Case ID	UC-6
Use Case Name	Add to Cart
Primary Actor	User
Description	Adds selected items to user's cart.
Trigger	User clicks "Add to Cart."
Preconditions	Product available in stock.
Postconditions	Item saved to cart.
Normal Flow	1. User adds item. 2. System stores product in cart.
Alternative Flows	Product unavailable → display message.
Exceptions	Session timeout before save.
Business Rules	Cart limit: 20 items per user.
Assumptions	Product stock updated in real-time.

3.3.7 UC-7: Review Cart

Allows users to view and edit products in their cart before placing an order.

Table 3.7: UC-7 Review Cart

Use Case ID	UC-7
Use Case Name	Review Cart
Primary Actor	User
Description	Shows all items added to the cart.
Trigger	User opens cart.
Preconditions	User must have items in cart.
Postconditions	Updated cart displayed.
Normal Flow	1. System lists items. 2. User edits/removes as needed.
Alternative Flows	Empty cart → “No items in cart.”
Exceptions	Temporary session loss.
Business Rules	Cart auto-cleared after 48 hours of inactivity.
Assumptions	Prices in cart reflect real-time discounts.

3.3.8 UC-8: Place Order

This use case completes the purchase process after reviewing the cart and confirming payment.

Table 3.8: UC-8: Place Order

Use Case ID	UC-8
Use Case Name	Place Order
Primary Actor	User
Description	Finalizes product purchase.
Trigger	User clicks “Confirm Order.”
Preconditions	Items exist in cart; user logged in.
Postconditions	Order stored in system.
Normal Flow	1. User confirms order. 2. System processes payment. 3. Order confirmed.
Alternative Flows	Payment failed → retry option.
Exceptions	Server crash → temporary pending order.
Business Rules	Order ID auto-generated.

Assumptions	Payment gateway functional.
-------------	-----------------------------

3.3.9 UC-9: View & Manage Orders

Allows Admin to view and update order statuses and details.

Table 3.9: UC-9: View & Manage Orders

Use Case ID	UC-9
Use Case Name	View & Manage Orders
Primary Actor	Admin
Description	Enables Admin to monitor and manage customer orders.
Trigger	Admin selects “Manage Orders.”
Preconditions	Orders must exist.
Postconditions	Updated order statuses saved.
Normal Flow	1. Admin views orders. 2. Updates status.
Alternative Flows	No orders found → show message.
Exceptions	Database query timeout.
Business Rules	Only Admin can change order state.
Assumptions	Orders updated in real-time.

3.3.10 UC-10: Track Order (Real-Time)

Allows users to check real-time status of their placed orders.

Table 3.10: UC-10: Track Order (Real-Time)

Use Case ID	UC-10
Use Case Name	Track Order (Real-Time)
Primary Actor	User
Description	Displays live order tracking and delivery progress.
Trigger	User selects “Track Order.”
Preconditions	Order placed successfully.
Postconditions	Live order data displayed.
Normal Flow	1. User views tracking. 2. System shows current status.

Alternative Flows	Tracking delayed → message shown.
Exceptions	Delivery API offline.
Business Rules	Tracking updates every 10 minutes.
Assumptions	Courier system integrated.

3.3.11 UC-11: Modify / Cancel Order

Allows users to change or cancel an order before dispatch.

Table 3.11: UC-11: Modify / Cancel Order

Use Case ID	UC-11
Use Case Name	Modify / Cancel Order
Primary Actor	User
Description	Users can update or cancel pending orders.
Trigger	User selects “Cancel/Modify Order.”
Preconditions	Order not yet dispatched.
Postconditions	Order updated or removed.
Normal Flow	1. User edits or cancels. 2. System updates database.
Alternative Flows	Dispatched orders → cancel disabled.
Exceptions	System lag → delayed update.
Business Rules	Modification allowed within 30 minutes of order.
Assumptions	User logged in during update.

3.3.12 UC-12: Accept / Dispatch Orders

Admin updates the order status to accepted, dispatched, or delivered.

Table 3.12: UC-12: Accept / Dispatch Orders

Use Case ID	UC-12
Use Case Name	Accept / Dispatch Orders
Primary Actor	Admin
Description	Admin controls the dispatch process.
Trigger	Admin selects an order.
Preconditions	Order must exist.
Postconditions	Updated order status saved.

Normal Flow	1. Admin approves order. 2. System updates status.
Alternative Flows	Invalid status change → show alert.
Exceptions	Database lock during update.
Business Rules	Orders dispatched FIFO basis.
Assumptions	Delivery partner available.

3.3.13 UC-13: Wishlist Management

This use case allows users to add or remove items from their wishlist.

Table 3.13: UC-13: Wishlist Management

Use Case ID	UC-13
Use Case Name	Wishlist Management
Primary Actor	User
Description	Manages user wishlist for future purchases.
Trigger	User clicks “Add to Wishlist.”
Preconditions	User logged in.
Postconditions	Wishlist updated.
Normal Flow	1. User adds/removes product. 2. System saves change.
Alternative Flows	Product already in wishlist.
Exceptions	Network timeout.
Business Rules	Wishlist holds up to 50 items.
Assumptions	Wishlist syncs across devices.

3.3.14 UC-14: Provide Feedback & Rating

This use case allows customers to provide ratings and feedback after successful order delivery.

Table 3.14: UC-14: Provide Feedback & Rating

Use Case ID	UC-14
Use Case Name	Provide Feedback & Rating
Primary Actor	User
Description	Lets users rate and comment on products and service.
Trigger	Order marked “Delivered.”
Preconditions	Order completed successfully.

Postconditions	Feedback saved in database.
Normal Flow	1. System prompts rating. 2. User submits feedback. 3. Data stored.
Alternative Flows	User skips → “No Feedback.”
Exceptions	Server error → cache temporarily.
Business Rules	Feedback editable for 24 hours.
Assumptions	Users provide honest reviews.

3.3.15 UC-15: Add Flash Offers / Discounts

This use case allows Admin to add promotional discounts or flash offers.

Table 3.15: UC-15: Add Flash Offers / Discounts

Use Case ID	UC-15
Use Case Name	Add Flash Offers / Discounts
Primary Actor	Admin
Description	Admin defines limited-time deals for customers.
Trigger	Admin selects “Add Offer.”
Preconditions	Admin logged in.
Postconditions	Offer applied to selected products.
Normal Flow	1. Admin enters offer details. 2. System applies discount.
Alternative Flows	Invalid offer → show warning.
Exceptions	Offer expiry time not set.
Business Rules	Offer auto-expires after deadline.
Assumptions	Offer applied system-wide instantly.

3.3.16 UC-16: View Reports & Earnings

This use case enables Admin to view system-generated sales, earnings, and user analytics.

Table 3.16: UC-16: View Reports & Earnings

Use Case ID	UC-16
Use Case Name	View Reports & Earnings
Primary Actor	Admin
Description	Displays summarized reports and insights.

Trigger	Admin selects “View Reports.”
Preconditions	Orders and user data must exist.
Postconditions	Reports displayed successfully.
Normal Flow	1. Admin opens reports section. 2. System fetches and displays data.
Alternative Flows	No data → show “Empty Report.”
Exceptions	Analytics service unavailable.
Business Rules	Reports update daily at midnight.
Assumptions	Data stored accurately.

3.3.17 UC-17: Add / Manage Users

This use case enables Admin to add, edit, or delete system users.

Table 3.17: UC-17: Add / Manage Users

Use Case ID	UC-17
Use Case Name	Add / Manage Users
Primary Actor	Admin
Description	Admin manages system users.
Trigger	Admin selects “Manage Users.”
Preconditions	Admin logged in.
Postconditions	Updated user list saved.
Normal Flow	1. Admin edits or adds users. 2. System saves changes.
Alternative Flows	Invalid input → show error.
Exceptions	Database conflict on deletion.
Business Rules	Each user must have unique ID.
Assumptions	Admin performs valid operations.

3.4 Functional Requirements

The functional requirements of Grocify – The Online Supermart define how the system and its users interact to achieve the objectives of efficient grocery shopping, order management, and platform administration. Each requirement focuses on the behavior, operations, and interactions that the system must perform to meet user needs.

3.4.1 FR-1 User Management & Authentication

This function allows users to securely register, log in, and manage their accounts. It uses encrypted credentials and token-based authentication to ensure privacy and secure access. The feature also includes password recovery and account management options for ease of use.

Table 3.18: FR-1 User Management & Authentication

Feature ID	FR-1
Feature Name	User Management & Authentication
Description	Enables customers, vendors, and admins to register, log in, and manage their accounts securely using email or mobile number. Includes password reset, validation, and secure session management.
Inputs	Email/Phone, Password, Verification Code
Outputs	Access to user dashboard upon successful authentication
Source	User, Admin
Constraints	Must comply with secure password and encryption policies
Dependencies	Database, Authentication Server
Priority	High

3.4.2 FR-2 Product Browsing & Smart Search

Users can browse products across various categories and search using keywords or filters. The system provides real-time search results and supports multilingual input (Urdu + English). It ensures fast, accurate, and user-friendly product discovery.

Table 3.19: FR-2 Product Browsing & Smart Search

Feature ID	FR-2
Feature Name	Product Browsing & Smart Search
Description	Allows customers to browse grocery items by category and search via keywords or voice (Urdu & English). Supports filters, sorting, and displays detailed product info.
Inputs	Search Query, Voice Input, Category Filter
Outputs	List of products with details (price, image, stock status)

Source	Customer
Constraints	Requires stable internet connection
Dependencies	Product Database, Search Engine API
Priority	High

3.4.3 FR-3 Shopping Cart & Order Placement

This function enables users to add, remove, or update items in their cart and proceed with checkout securely. It includes order summary, payment options, and confirmation screens. The process ensures transparency and smooth user experience.

Table 3.20: FR-3 Shopping Cart & Order Placement

Feature ID	FR-3
Feature Name	Shopping Cart & Order Placement
Description	Allows customers to add, edit, or remove items in the cart, calculate total price, and confirm order checkout with unique order ID generation.
Inputs	Selected Products, Quantity, Payment Method
Outputs	Order Confirmation Message and Order ID
Source	Customer
Constraints	Requires valid payment and network connection
Dependencies	Payment Gateway, Database
Priority	High

3.4.4 FR-4 Order Tracking & Management

Users can track their orders in real time using GPS integration. Notifications keep them informed at every stage order confirmation, dispatch, and delivery. This ensures transparency and builds user trust in the delivery process.

Table 3.21: FR-4 Order Tracking & Management

Feature ID	FR-4
Feature Name	Order Tracking & Management

Description	Provides real-time order tracking via GPS. Customers can view delivery status, vendors can update progress, and admins monitor order flow.
Inputs	Order ID, Tracking Request
Outputs	Live order status and delivery time updates
Source	Customer, Vendor, Admin
Constraints	GPS and Internet must be active
Dependencies	Google Maps API, Delivery System
Priority	High

3.4.5 FR-5 Product & Inventory Management

Before dispatch, users can modify quantities or cancel orders through an easy interface. The system updates the inventory and notifies vendors immediately. This adds flexibility and minimizes customer dissatisfaction.

Table 3.22: FR-5 Product & Inventory Management

Feature ID	FR-5
Feature Name	Product & Inventory Management
Description	Enables vendors and admins to add, edit, or remove product listings. Automatically updates inventory after each order.
Inputs	Product Name, Price, Description, Image, Stock Quantity
Outputs	Updated product catalog and inventory data
Source	Vendor, Admin
Constraints	Product information must be complete
Dependencies	Product Database, Admin Panel
Priority	Medium

3.4.6 FR-6 Feedback & Rating System

This feature allows users to save preferred items for future purchases. It simplifies repeat shopping and sends alerts when saved items are restocked or discounted. It enhances convenience and improves user engagement.

Table 3.23: FR-6 Feedback & Rating System

Feature ID	FR-6
Feature Name	Feedback & Rating System
Description	Lets customers rate their orders, products, and delivery service, and submit feedback for service improvement.
Inputs	Rating (1–5), Review Text
Outputs	Stored rating and feedback summary
Source	Customer
Constraints	One feedback per completed order
Dependencies	Feedback Database, Reporting Module
Priority	Medium

3.4.7 FR-7 Smart Notifications & Promotions

Users can set monthly budgets, track spending, and view category-wise expenses. AI provides insights and suggestions for cost-effective shopping. This feature promotes financial awareness and spending control.

Table 3.24: FR-7 Smart Notifications & Promotions

Feature ID	FR-7
Feature Name	Smart Notifications & Promotions
Description	Sends automated, AI-driven notifications about discounts, order status updates, and restocked items. Admin can also broadcast messages.
Inputs	Event Triggers (Order Placed, Offer Created)
Outputs	Push Notification or In-App Alert
Source	System, Admin
Constraints	Notifications should be non-repetitive and relevant
Dependencies	AI Engine, Notification API
Priority	Medium

3.4.8 FR-8 Budgeting & Expense Tracking

The system analyzes past purchases, search history, and preferences to recommend suitable products. It provides personalized suggestions in real time. This improves user engagement and encourages repeat orders.

Table 3.25: FR-8 Budgeting & Expense Tracking

Feature ID	FR-8
Feature Name	Budgeting & Expense Tracking
Description	Allows users to set monthly budgets, track grocery expenses, and receive alerts for overspending. Generates monthly reports.
Inputs	Monthly Budget Limit, Order History
Outputs	Expense Report, Budget Alert
Source	Customer
Constraints	Requires active order data
Dependencies	Expense Tracking Module, Database
Priority	Low

3.4.9 FR-9 Recipe & Product Integration

The AI-powered chatbot provides 24/7 support for user queries, order updates, and troubleshooting. It understands natural language inputs and can escalate issues to human support if required. This ensures faster responses and improved user satisfaction.

Table 3.26: FR-9 Recipe & Product Integration

Feature ID	FR-9
Feature Name	Recipe & Product Integration
Description	Integrates recipes with grocery shopping. Suggests dishes and allows users to add all ingredients to the cart with one click.
Inputs	Selected Recipe, Serving Size
Outputs	Ingredient List Added to Cart
Source	Customer
Constraints	Ingredients must exist in product database
Dependencies	Recipe Database, Product Catalog
Priority	Low

3.4.10 FR-10 Admin Dashboard & Control

The admin dashboard enables control over users, products, and orders. Admins can add, edit, or delete records, track performance, and view total earnings. It ensures efficient management and smooth system operation.

Table 3.27: FR-10 Admin Dashboard & Control

Feature ID	FR-10
Feature Name	Admin Dashboard & Control
Description	Provides admins a centralized dashboard to manage users, vendors, products, and orders. Displays analytics for earnings and platform performance.
Inputs	Admin Commands, System Data
Outputs	Dashboard Reports, Updated Records
Source	Admin
Constraints	Accessible only with admin privileges
Dependencies	Admin Panel, Database, Reporting Engine
Priority	High

3.5 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints that the Grocify system must satisfy, independent of specific functional behaviors. These requirements are derived from the operational context, user expectations, and platform standards documented in the SRS.

3.5.1 Usability Requirements

The system shall be designed to minimize the learning curve for new users. A first-time user with no prior experience of similar applications shall be able to complete a grocery order within five interactions from the home screen, as validated through usability testing. The application shall provide complete bilingual support for English and Urdu across all primary user-facing screens, including product names, error messages, notification text, and navigation labels. All input forms shall provide real-time inline validation with descriptive error messages that clearly indicate the nature of the error and the corrective action required. All irreversible actions including order placement, order cancellation, and account deletion shall require explicit user confirmation through modal dialogs before execution.

3.5.2 Performance Requirements

Performance benchmarks for the Grocify system are as follows. At least ninety-five percent of all application screens shall render completely within three seconds when the device is connected to a standard 4G mobile internet connection [8]. Product search queries shall return results within two seconds of submission. The backend API shall handle a minimum of one thousand concurrent user connections without response time degradation exceeding fifty percent of baseline. Push notification delivery shall occur within five seconds of the triggering event. The checkout and order placement process shall complete within eight seconds end-to-end under standard load conditions.

3.5.3 Security Requirements

All user passwords shall be hashed using the bcrypt algorithm with a minimum cost factor of twelve before storage in the database. No plaintext passwords shall be stored or transmitted at any point. All client-server communication shall be encrypted using HTTPS with TLS 1.3 or higher [9]. The system shall implement JWT-based session management with a token expiry period of twenty-four hours. All API endpoints that access or modify user data or system state shall be protected by JWT authentication middleware. Input validation shall be applied to all user-supplied data at both the client and server levels to prevent SQL injection, cross-site scripting (XSS), and other injection attacks. Role-based access control shall ensure that Customers cannot access Vendor or Admin functionality, and that Vendors cannot access Admin-exclusive functionality.

3.5.4 Reliability Requirements

The system shall maintain a minimum uptime of ninety-nine percent during standard operational hours (6:00 AM to 11:00 PM). Automated MySQL database backups shall be performed every twenty-four hours and stored in a geographically separate location. The application shall implement graceful degradation, maintaining core functionality (order history viewing, status checking) in the event of partial backend unavailability. The system shall recover from unexpected network disconnections within ten seconds of reconnection and shall resume interrupted operations where possible.

3.5.5 Scalability Requirements

The system architecture shall be designed to support horizontal scaling of the Node.js backend through load balancing, allowing additional server instances to be added without

system reconfiguration. The MySQL database shall be capable of efficiently managing a minimum of five hundred thousand product records and ten million order records. The backend API shall support integration with additional third-party services including alternative payment gateways and logistics management APIs without requiring fundamental architectural changes.

3.5.6 Portability Requirements

The Android mobile application shall be compatible with all Android devices running version 8.0 (Oreo) or later, covering a substantial majority of Android devices currently in use in Pakistan. The application's user interface shall be fully responsive across standard screen sizes ranging from 5.0 inches to 7.0 inches. The backend server shall be deployable on both Linux and Windows server environments, and shall be compatible with cloud hosting platforms including Amazon Web Services (AWS) and Google Cloud Platform.

CHAPTER 4

DESIGN AND ARCHITECTURE

4 DESIGN AND ARCHITECTURE

This chapter presents the complete system design and architectural decisions for Grocify – The Online Supermart. The design was developed using Object-Oriented Programming principles and documented through a suite of standard UML modeling artifacts. The chapter covers the overall system architecture, the data representation model including the entity-relationship schema and data dictionary, the process flow representations for the five primary system workflows, and the UML design models including the class diagram, sequence diagrams, state transition diagram, and data flow diagrams at multiple levels of abstraction.

4.1 System Architecture

Grocify is built on a three-tier client-server architecture, a well-established architectural pattern in enterprise application development that provides clean separation between the user interface, business logic, and data storage concerns [10]. This architectural decision was motivated by the need to support independent development, testing, and scaling of each layer, to enforce a clear security boundary between the client application and the data store, and to facilitate future expansion of the system including web client development and third-party API integrations without requiring changes to the core data layer.

4.1.1 Presentation Layer

The presentation layer is implemented as a native Android mobile application developed using Kotlin in Android Studio. The application follows the Model-View-ViewModel (MVVM) architectural pattern at the intra-layer level, providing a clean separation between the UI components (Activities and Fragments), the UI state management (ViewModels), and the data access logic (Repository classes). This pattern was selected for its compatibility with Android Jetpack components, its inherent testability, and its support for reactive UI updates through LiveData and Kotlin Coroutines.

The user interface is constructed using Android's native View system with RecyclerView adapters for efficient list rendering, ConstraintLayout for complex UI compositions, and Material Design components for visual consistency. Navigation between screens is

managed by the Android Jetpack Navigation Component with a single-activity architecture. The Google Maps SDK for Android is integrated at this layer to render the live delivery tracking map.

4.1.2 Application Layer

The application layer is implemented as a Node.js RESTful API server using the Express.js framework. This layer is responsible for all business logic processing, API request routing, input validation, authentication middleware, and coordination between the frontend client and the data layer. The selection of Node.js was motivated by its event-driven, non-blocking I/O architecture, which is well-suited to handling the high volumes of concurrent, short-duration API requests characteristic of mobile commerce applications [11].

The backend service is organized into a layered module structure comprising route definitions, controller functions, service layer functions, and data access objects (DAOs). Authentication is enforced at the route level through JWT middleware that validates tokens, extracts the user's identity and role, and passes this information to downstream controllers. FCM notifications are dispatched asynchronously by a dedicated notification service module to avoid blocking the primary request handling pipeline.

4.1.3 Data Layer

The data layer consists of a MySQL relational database management system hosting the Grocify database schema. MySQL was selected for its maturity, reliability, comprehensive documentation, strong support for ACID (Atomicity, Consistency, Isolation, Durability) transactions, and broad compatibility with Node.js through the mysql2 driver library. The database schema is normalized to Third Normal Form (3NF) to eliminate data redundancy and ensure update anomalies cannot arise [12].

Indexing is applied to all primary key columns (automatically, by MySQL), and additionally to the frequently searched columns `Product.name`, `Product.category`, `Order.userId`, `Order.vendorId`, and `Order.status`. Foreign key constraints with appropriate cascade rules enforce referential integrity throughout the schema.

4.2 Data Representation

Data representation in the Grocify – Online Supermart system defines how data is structured, stored, and managed within the application. The system uses structured formats to represent entities such as users, products, orders, vendors, and transactions, where each entity is modeled as an object and mapped to relational database tables for persistent storage. Relationships like one-to-many and many-to-many (e.g., user-to-orders and order-to-products) ensure data integrity and efficient retrieval. On the mobile side, data is temporarily managed using local storage and ViewModels to maintain UI state and improve performance, while APIs transfer data in JSON format between client and server, enabling scalability, consistency, and efficient operations such as searching and real-time updates.

4.2.1 Entity-Relationship Model

The Grocify entity-relationship model comprises sixteen primary entities with well-defined relationships. The key entity relationships are as follows. The User entity is related to the Order entity through a one-to-many relationship: one User may place many Orders, but each Order belongs to exactly one User. This relationship is enforced through a foreign key on the Order.userId column referencing User.userId. The Vendor entity is related to the Product entity through a composition relationship: Products are existentially dependent on their owning Vendor, meaning that if a Vendor record is deleted, all associated Product records are cascade-deleted. The Order and Product entities share a many-to-many relationship, resolved through the OrderItem join entity, which stores the quantity ordered and the unit price at the time of purchase for each Product in each Order. The Cart entity belongs to exactly one User and contains multiple CartItem entities, each of which references one Product. When an order is placed, CartItems are converted to OrderItems and the Cart is cleared. The Inventory entity associates each Product with its Vendor, storing the current stock quantity and expiry date for perishable items. The Payment entity has a one-to-one relationship with Order, recording the payment method and transaction status for each completed order.

4.2.2 Data Dictionary

The data dictionary defines all database entities, their attributes, data types, and purposes. The sixteen entities are documented as follows.

Table 4.1: User Table

Field	Type	Constraints	Description
userId	INT	PRIMARY KEY, AUTO_INCREMENT	Unique system- generated user identifier
name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(150)	NOT NULL, UNIQUE	Login email; unique across all roles
password	VARCHAR(255)	NOT NULL	bcrypt hash of user's password
phone	VARCHAR(15)	NOT NULL	Contact number for delivery coordination
address	VARCHAR(300)		Default delivery address
monthlyBudget	FLOAT	DEFAULT 0	User- defined monthly grocery spending limit in PKR

role	ENUM('User','Vendor','Admin')	NOT NULL, DEFAULT 'User'	Role-based access control identifier
------	-------------------------------	-----------------------------	---

Table 4.2: Order Table

Field	Type	Constraints	Description
orderId	INT	PRIMARY KEY, AUTO_INCREMENT	Unique auto-generated order identifier
userId	INT	FK → User.userId, NOT NULL	Customer who placed the order
vendorId	INT	FK → Vendor.vendorId, NOT NULL	Vendor fulfilling the order
totalAmount	FLOAT	NOT NULL	Total order value in PKR at time of placement
status	ENUM	NOT NULL, DEFAULT 'Pending'	Pending Confirmed Prepared Dispatched Delivered Cancelled
orderDate	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp of order placement
deliveryAddress	VARCHAR(300)	NOT NULL	Delivery address

			captured at order time
--	--	--	------------------------

Table 4.3: Product Table

Field	Type	Constraints	Description
productId	INT	PRIMARY KEY, AUTO_INCREMENT	Unique product identifier
vendorId	INT	FK → Vendor.vendorId, NOT NULL	Owning vendor; CASCADE DELETE
name	VARCHAR(200)	NOT NULL, INDEX	Product display name; indexed for search
category	VARCHAR(100)	NOT NULL, INDEX	Product category; indexed for browse/filter
price	FLOAT	NOT NULL, CHECK (price > 0)	Current selling price in PKR
description	TEXT		Detailed product information
imageUrl	VARCHAR(500)		Cloud storage URL for product image

Table 4.4: Inventory Table

Field	Type	Constraints	Description
inventoryId	INT	PRIMARY KEY, AUTO_INCREMENT	Unique inventory

			record identifier
vendorId	INT	FK → Vendor.vendorId, NOT NULL	Vendor who owns this inventory
productId	INT	FK → Product.productId, NOT NULL	Product being tracked
stock	INT	NOT NULL, DEFAULT 0, CHECK (stock >= 0)	Current available stock quantity
expiryDate	DATE		Expiration date for perishable items

Table 4.5: Payment Table

Field	Type	Constraints	Description
paymentId	INT	PRIMARY KEY, AUTO_INCREMENT	Unique payment transaction identifier
orderId	INT	FK → Order.orderId, NOT NULL, UNIQUE	One-to-one link to associated order
amount	FLOAT	NOT NULL	Payment amount in PKR
method	ENUM	NOT NULL	Cash Card Easypaisa JazzCash

status	ENUM	NOT NULL, DEFAULT 'Pending'	Success Pending Failed
--------	------	--------------------------------	----------------------------------

4.3 Process Flow Representation

The process flow of Grocify outlines how different system components interact to complete essential operations. It provides a high-level view of how user requests move through the application, how data is processed at different layers, and how the system responds in real time. These flows ensure that each module works in a synchronized and efficient manner, maintaining consistency across the platform.

The following major processes are modeled and represented using flow diagrams, sequence charts, and activity workflows:

4.3.1 Product Browsing and Smart Search Flow

The process begins when the Customer opens the home or Search screen and enters a keyword or uses voice input via the microphone icon. For voice input, the app records speech, sends it to the Google Speech-to-Text API, and uses the returned transcription as the search query. The backend then runs a MySQL LIKE query against Product.name and Product.category, appending any active filters (price range, category, discount availability) to the WHERE clause. Results are sorted by relevance and rendered in a RecyclerView list. Simultaneously, the recommendation service pulls the user's purchase history from the OrderItem table and displays personalized product suggestions in a horizontal scroll section above the search results.

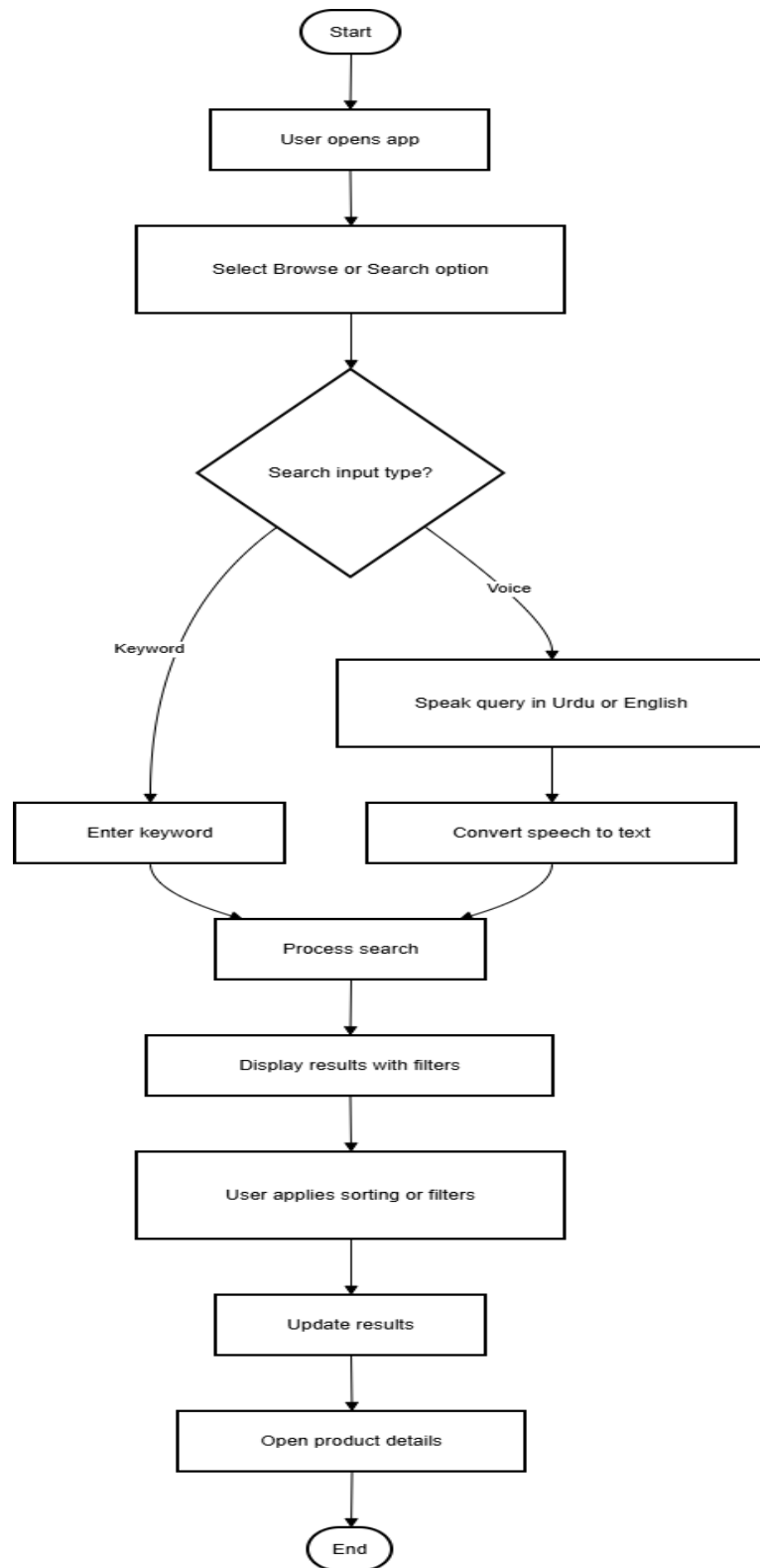


Figure 4.1: Product Browsing & Smart Search

4.3.2 Order Placement Flow

The process begins when the Customer taps 'Add to Cart' from the Product Detail screen. The Cart screen shows all added items with quantities and prices, which the Customer can adjust before tapping 'Checkout.' The Order Confirmation screen then displays the full order summary subtotal, delivery fee, taxes, voucher field, and total payable. After selecting a payment method and tapping 'Place Order,' the app sends a POST request to `/api/orders` with the cart contents, delivery address, and payment method. The backend validates the JWT token, then executes a single database transaction covering stock validation, Order and OrderItem record creation, inventory update, payment recording, and cart clearance. An FCM notification is dispatched to the Vendor and a success response with the order ID is returned to the app, which displays the Order Confirmation screen.

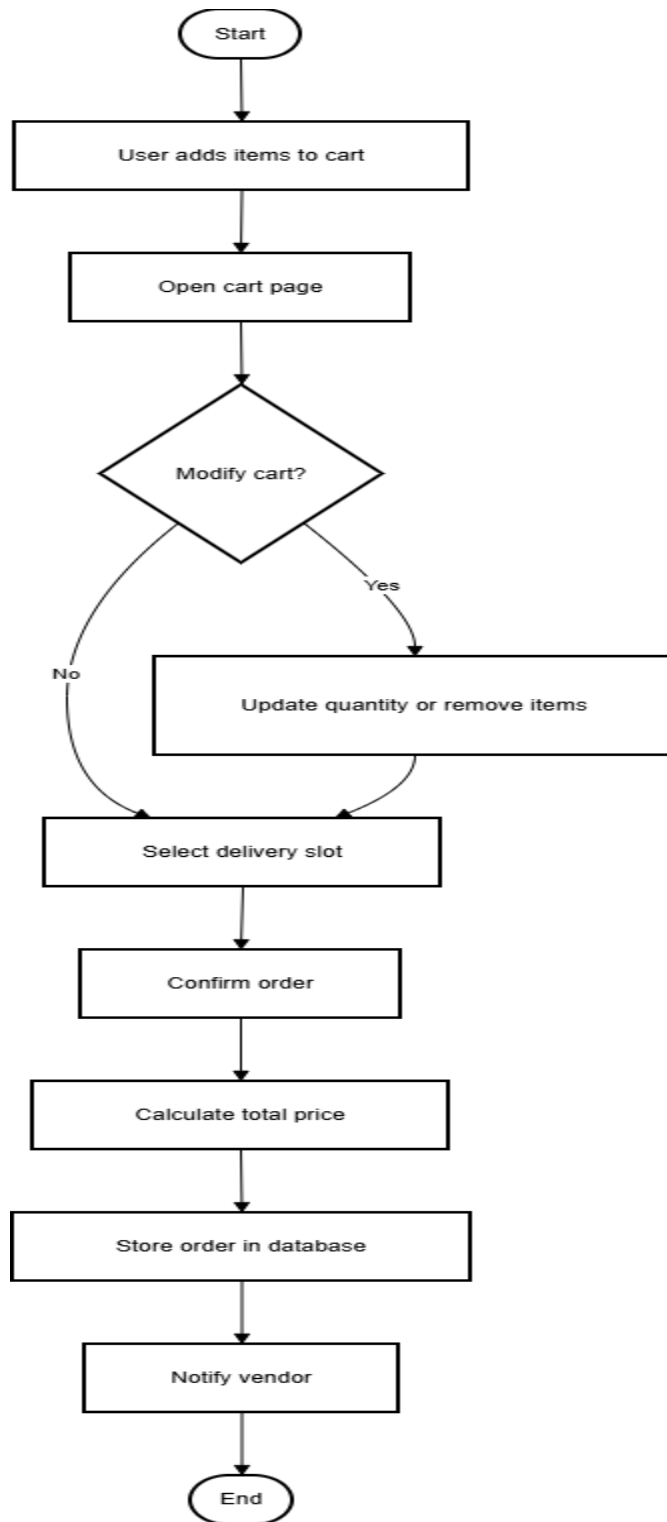


Figure 4.2: Order Placement

4.3.3 Real-Time Order Tracking Flow

Once an Administrator approves an order for dispatch, the order status in the database is updated to 'Dispatched' and the Customer receives an FCM push notification. The Customer taps 'Track Order' either from the notification or from the order history. The application makes a GET request to `/api/orders/:orderId/tracking`. The backend returns the current order status and, if the order is dispatched, the rider's most recently recorded GPS coordinates. The application renders a Google Maps view centered on the rider's location with a marker at the Customer's delivery address. The application polls the backend for updated coordinates every thirty seconds. As the rider progresses toward the delivery address, the map updates automatically. When the rider's recorded position is within fifty meters of the delivery address geofence, the backend automatically updates the order status to 'Delivered,' dispatches a delivery confirmation FCM notification, and prompts the Customer to submit feedback.

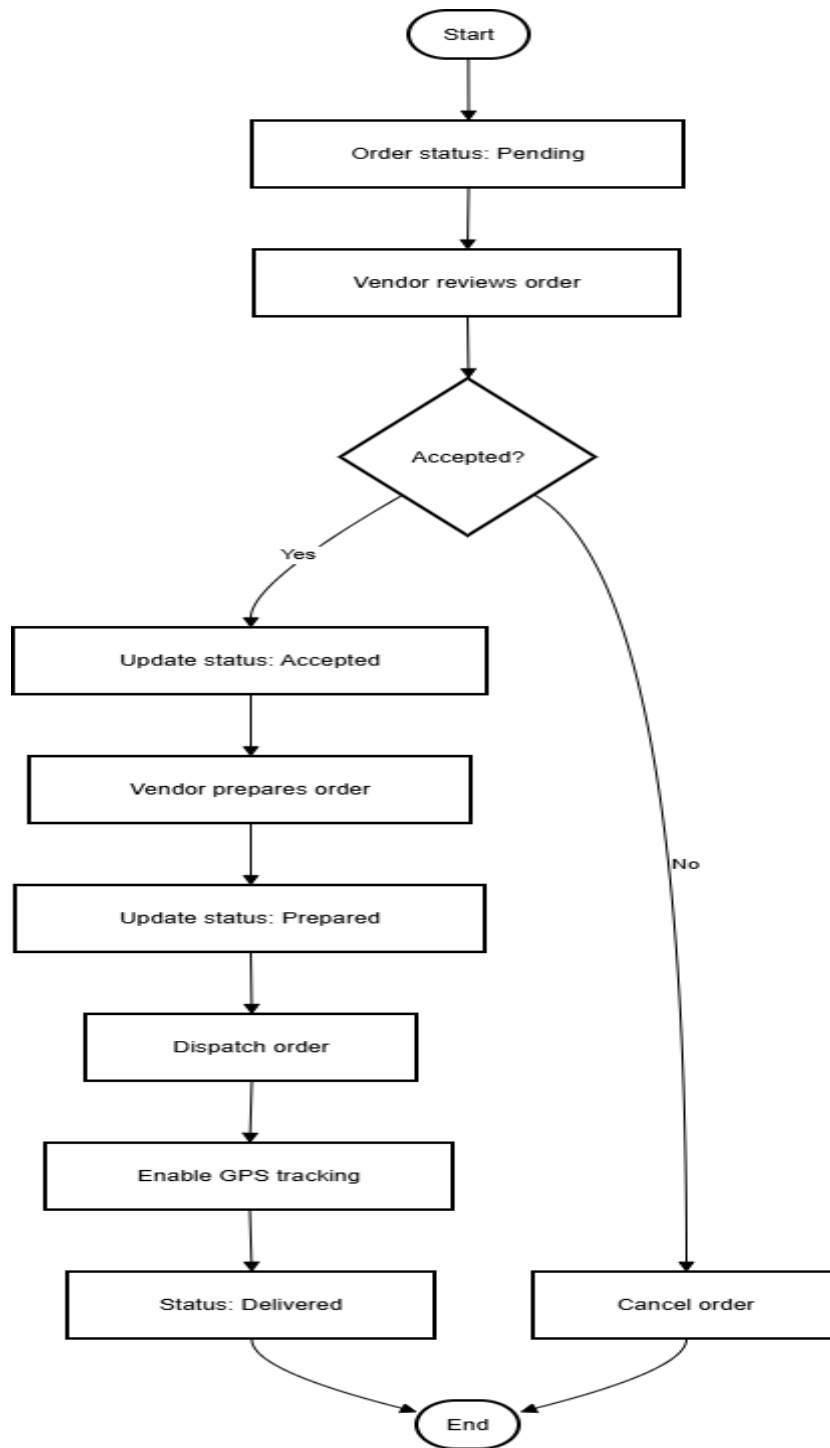


Figure 4.3: Order Tracking

4.3.4 Vendor Inventory Update Flow

The Vendor accesses the inventory management section of the Vendor Dashboard. The dashboard displays a list of the Vendor's current product catalog with current stock levels highlighted. The Vendor selects a product to update and is presented with an editable form pre-populated with the product's current details. After making modifications to the price, stock level, description, or image, the Vendor taps 'Save.' The application sends a PUT request to `/api/vendor/products/:productId` with the modified fields. The backend validates the Vendor's JWT token, confirms that the product belongs to the authenticated Vendor, validates all input fields, and updates the Product and Inventory tables. If the updated stock quantity is zero, the product's availability status is set to 'Out of Stock.' The backend returns a success response and the Vendor Dashboard refreshes to reflect the changes.

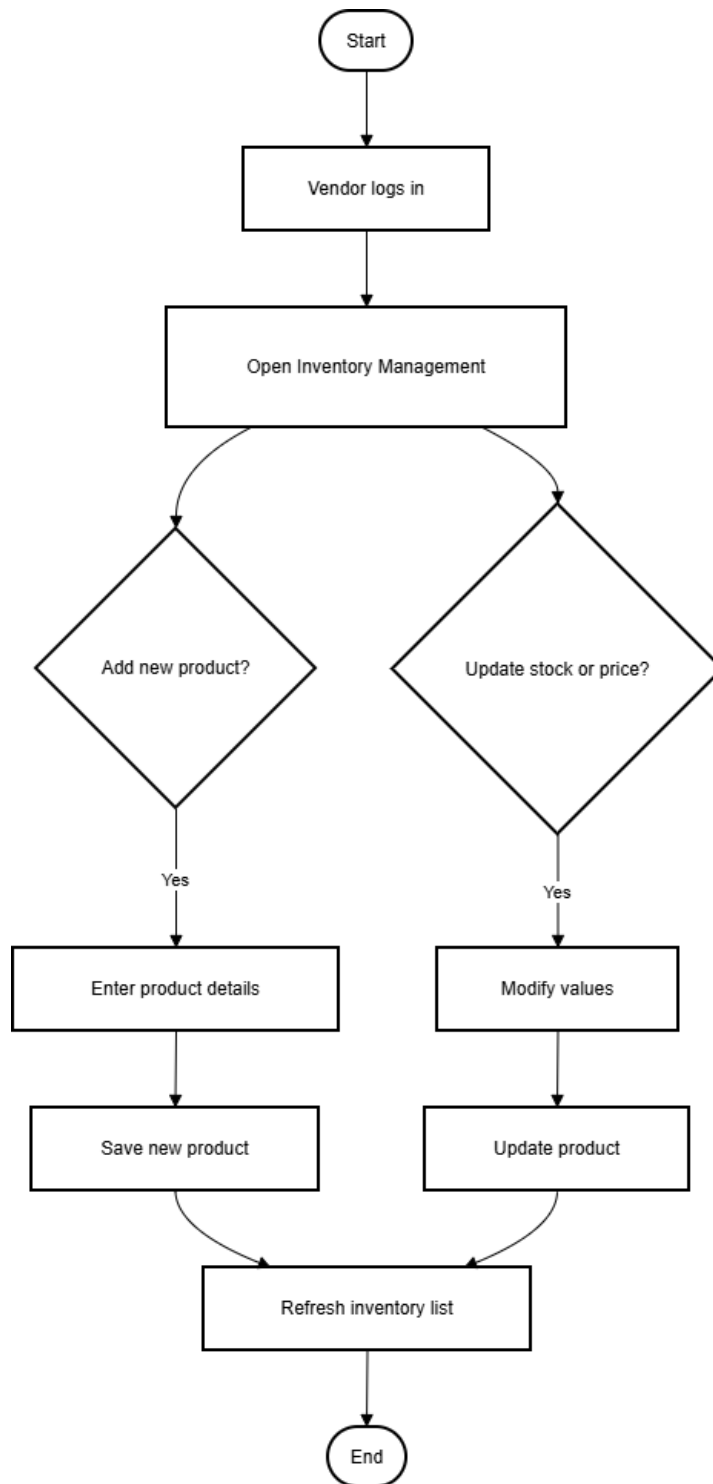


Figure 4.4: Vendor Inventory Update

4.3.5 Admin Dispatching Flow

The Administrator opens the Order Management section of the Admin Dashboard, which displays all pending orders sorted by placement time in ascending order (FIFO). Each order listing displays the order ID, Customer name, total amount, item count, and current status. The Administrator reviews a pending order, confirms the dispatch details are correct, and taps 'Approve Dispatch.' The application sends a PATCH request to `/api/admin/orders/:orderId/dispatch`. The backend validates the Administrator's JWT token, verifies that the order status is 'Confirmed' or 'Prepared,' updates the status to 'Dispatched' in the Order table, records the dispatch timestamp, dispatches an FCM push notification to the Customer, and returns a success response. The order list in the Admin Dashboard updates to reflect the new status.

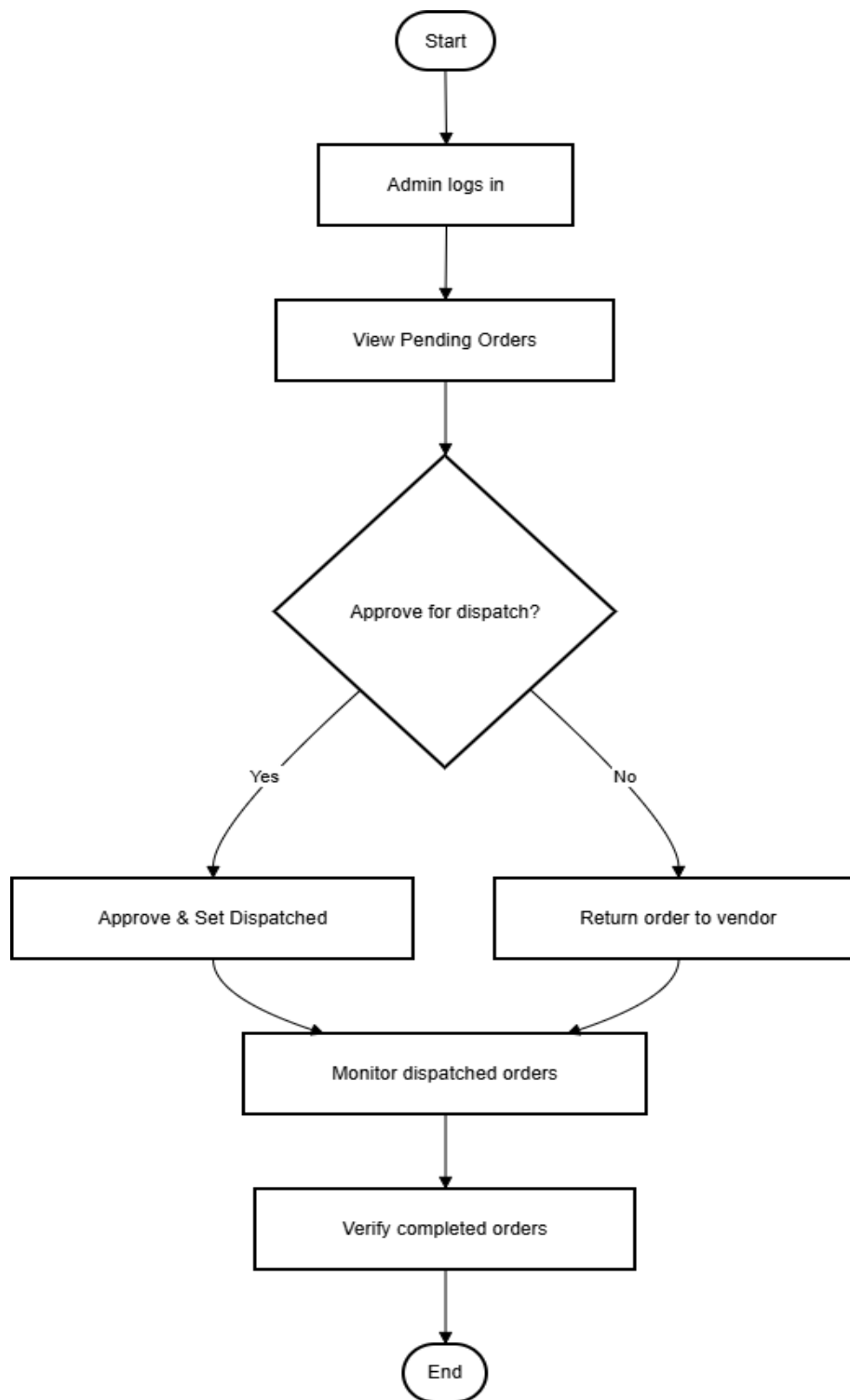


Figure 4.5: Admin Dispatching

4.4 Design Models

This section provides the visual representation of the system design using standard UML modeling techniques. These models illustrate how different components interact, how data flows through the system, and how system behavior changes based on events. The diagrams ensure that the system structure and functionality are clearly understood before implementation.

4.4.1 Class Diagram

The Grocify class diagram represents the structural design of the system using ten primary classes. The User class is the central entity of the system, containing attributes for `userId`, `name`, `email`, `password`, `phone`, `address`, `monthlyBudget`, and `role`, with methods for `register()`, `login()`, `updateProfile()`, and `setMonthlyBudget()`. The User class is connected to the Order class through a one-to-many aggregation relationship, representing the fact that a User can place multiple Orders but an Order cannot exist without an associated User.

The Vendor class inherits the authentication attributes from User (`email`, `password`) and adds `vendorName`, `shopName`, and `location` attributes specific to the vendor role. The Vendor class is connected to the Product class through a composition relationship, indicating that Products are existentially dependent on their Vendor. The Product class contains attributes for `productId`, `name`, `category`, `price`, `description`, and `imageUrl`, with methods for `updateDetails()`, `checkAvailability()`, and `applyDiscount()`.

The Order class contains the order lifecycle state machine, with attributes for `orderId`, `totalAmount`, `status`, `orderDate`, and `deliveryAddress`, and methods for `placeOrder()`, `modifyOrder()`, `cancelOrder()`, and `updateStatus()`. The Order class is connected to the OrderItem class through a composition relationship. The OrderItem class resolves the many-to-many relationship between Orders and Products, storing `quantity` and `priceAtPurchase` for each line item.

The Cart and CartItem classes model the pre-checkout state of a user's product selection. The Payment class records financial transaction details for each order, including the payment method and transaction status. The Inventory class manages stock levels for each Vendor-Product combination, with methods for `updateStock()`, `checkStock()`, and `alertLowStock()`. The Admin class extends User with administrative methods for `manageUsers()`, `dispatchOrders()`, and `viewReports()`.

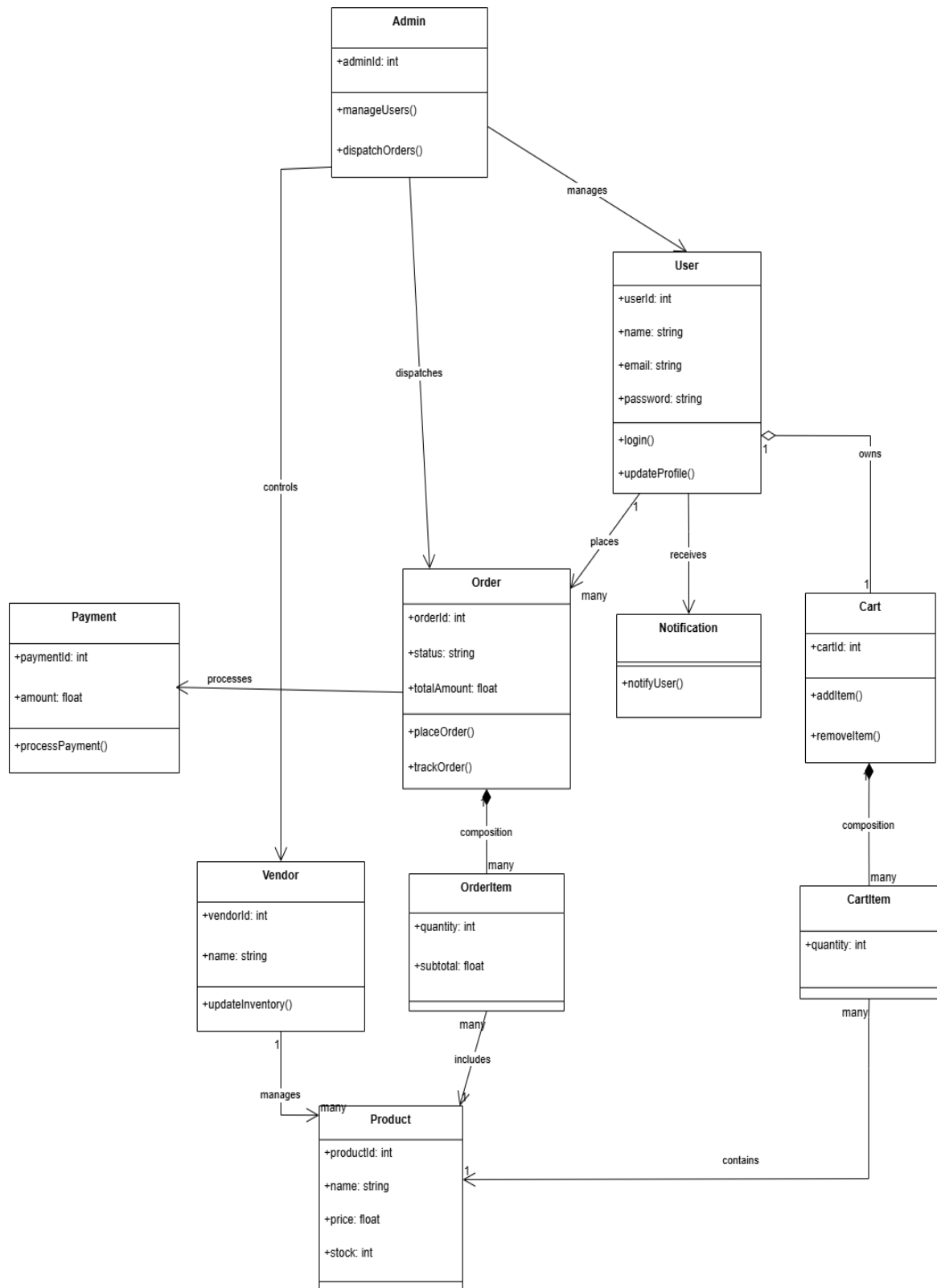


Figure 4.6: Class Diagram

4.4.2 Sequence Diagrams

Three sequence diagrams were developed to model the temporal interaction flows between actors and system components for the three most complex workflows in the system.

Place Order

The Place Order sequence diagram models the interaction between the Customer actor, the Android application (UI layer), the Node.js backend API (application layer), and the MySQL database (data layer). The diagram shows the Customer initiating the sequence by tapping 'Confirm Order,' the application sending a POST /api/orders request with the cart payload, the backend executing a multi-step database transaction (stock validation, order creation, inventory decrement, cart clearance), the backend dispatching an FCM notification to the Vendor, and the backend returning a success response with the order ID that the application displays in the confirmation screen.

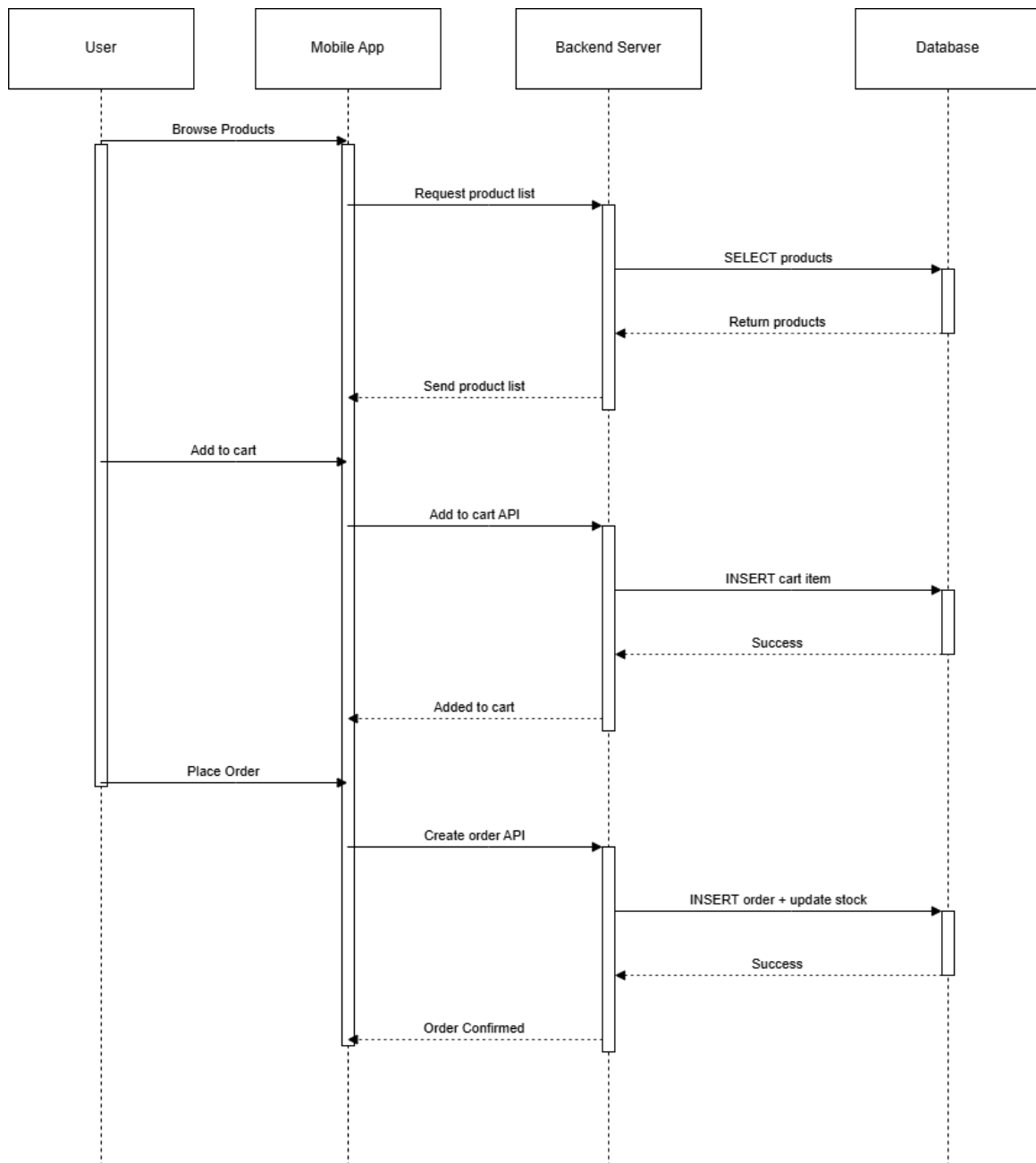


Figure 4.7: Place Order

Vendor Updating Inventory

The Vendor Updating Inventory sequence diagram shows the Vendor selecting a product in the Vendor Dashboard, the application retrieving the current product details from the backend, the Vendor submitting modified details, the backend validating and committing the changes to both the Product and Inventory tables, and the confirmation response being displayed to the Vendor.

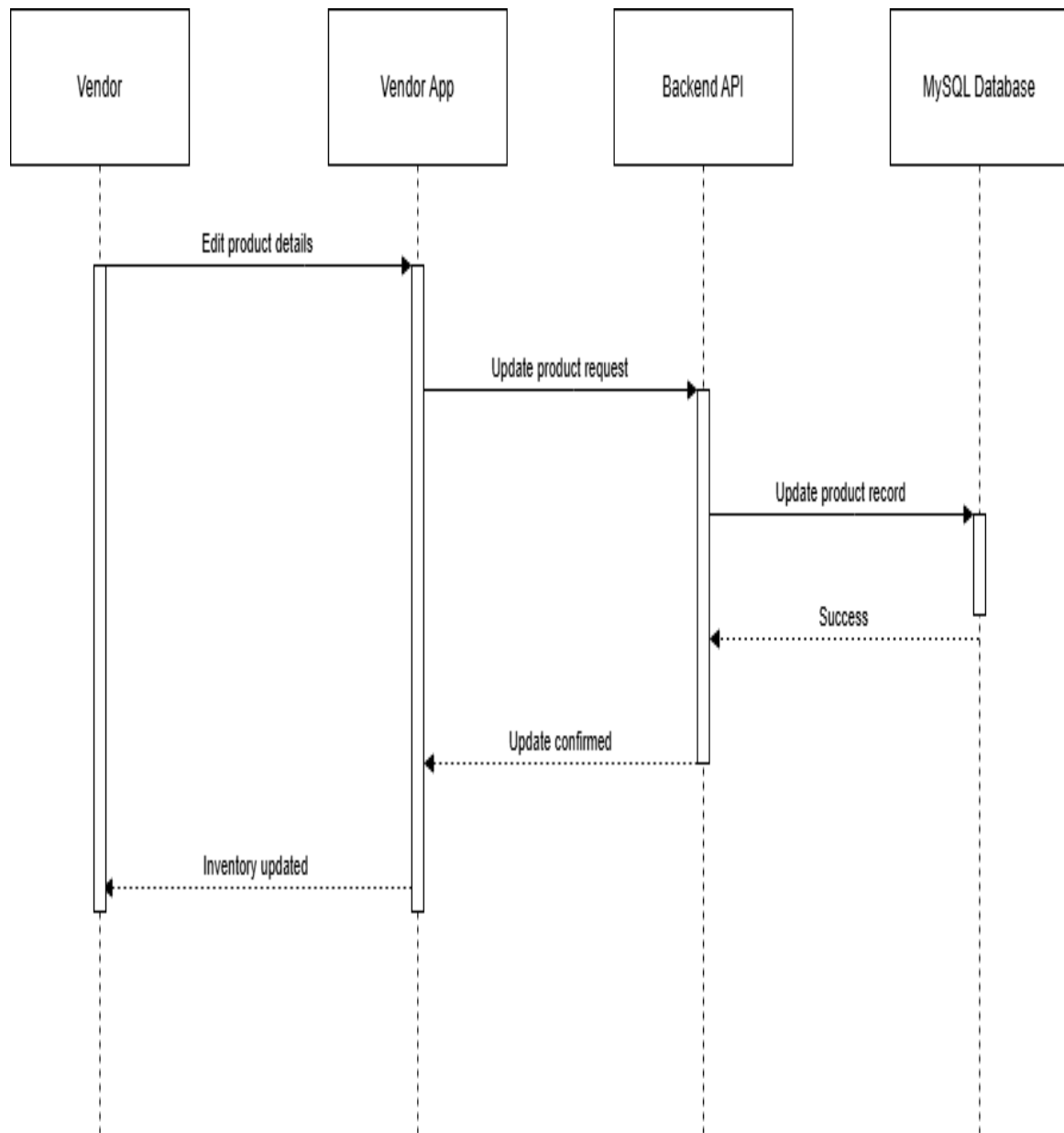


Figure 4.8: Vendor Updating Inventory

Admin Dispatching

The Admin Dispatching sequence diagram illustrates the admin accessing the order management interface, the application requesting pending orders from the backend, the backend querying the database for orders with eligible statuses, the Admin approving dispatch, the backend updating the order status and dispatching an FCM notification to the Customer, and the dashboard refreshing to reflect the updated order state.

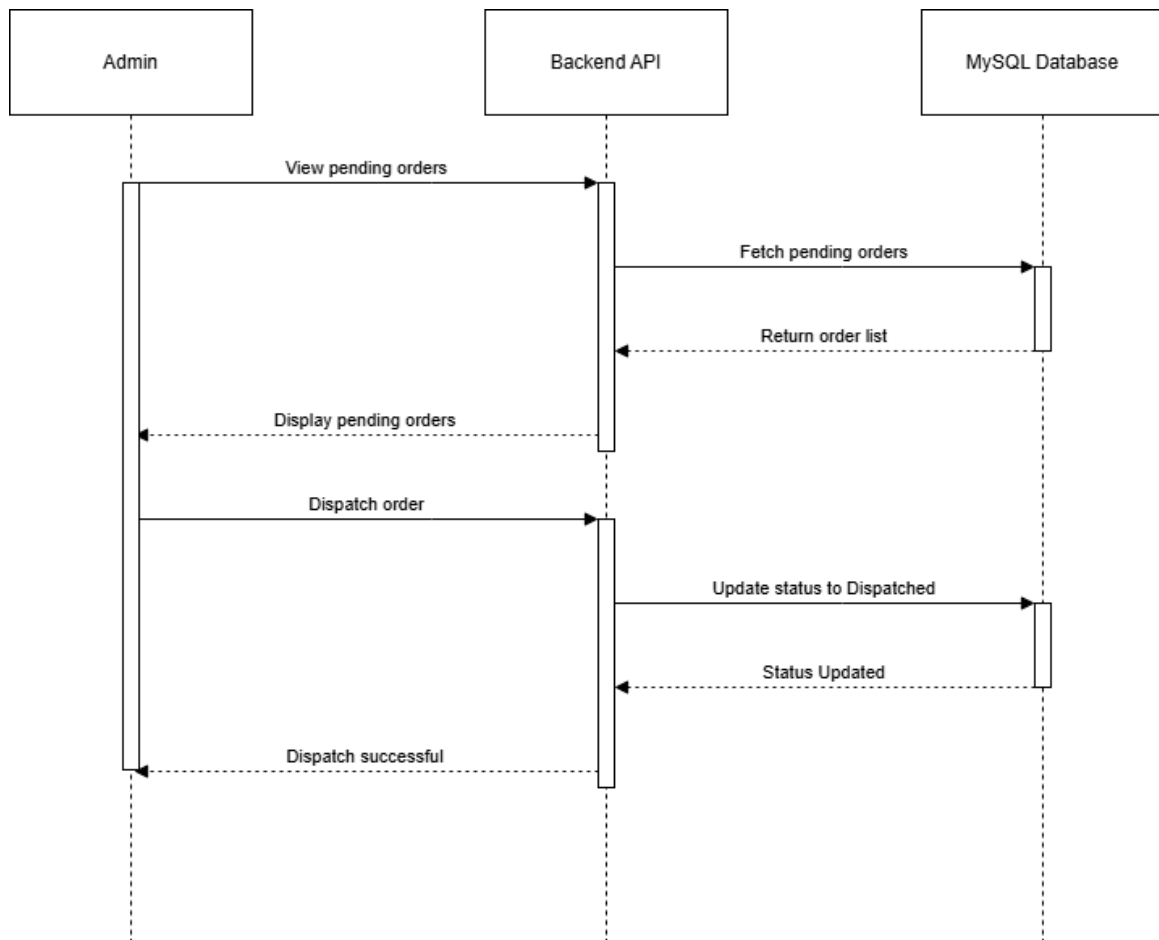


Figure 4.9: Admin Dispatching

4.4.3 State Transition Diagram — Order Lifecycle

The Order Lifecycle State Transition Diagram models all possible states of an Order entity and the valid transitions between them. An Order begins in the 'Pending' state immediately upon placement. The Administrator reviews and accepts the order, transitioning it to 'Confirmed.' The Vendor prepares the order for dispatch, transitioning it to 'Prepared.' The Administrator approves dispatch, transitioning the order to 'Dispatched.' The delivery rider delivers the order to the Customer, and the system automatically transitions the order to 'Delivered' based on GPS geofence confirmation or manual vendor confirmation.

The 'Cancelled' state is reachable from 'Pending' or 'Confirmed' through a Customer-initiated cancellation request, or from any pre-dispatch state through an Admin-initiated rejection. Once an order reaches the 'Dispatched' or 'Delivered' state, no further state transitions are possible. This state machine ensures that the order lifecycle is strictly controlled and that invalid state transitions cannot occur, protecting inventory integrity and financial record accuracy.

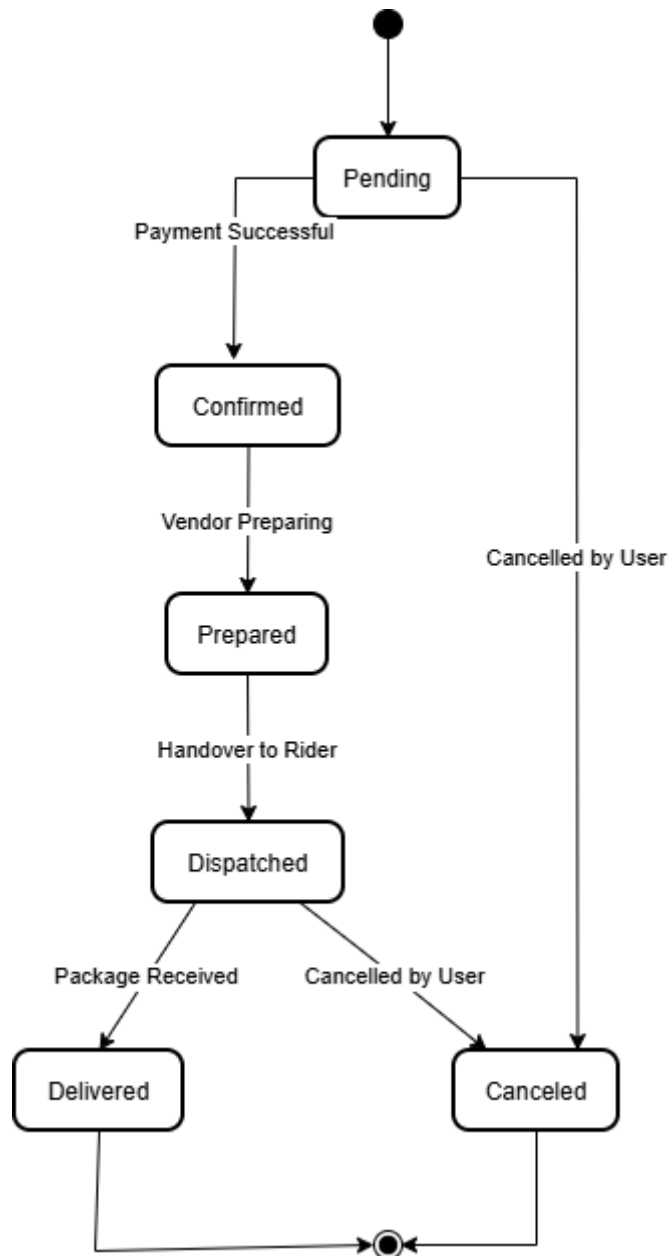


Figure 4.10: Order Lifecycle

4.4.4 Data Flow Diagrams

Data Flow Diagrams (DFDs) were developed at three levels of decomposition to progressively reveal the internal data processing logic of the Grocify system.

DFD – Level 0

The Level 0 DFD (Context Diagram) represents the entire Grocify system as a single process bubble interacting with three external entities: Customer, Vendor, and Administrator. Data flows at this level are high-level: the Customer sends search queries, cart actions, and order requests to the system, and receives product lists, order

confirmations, and delivery tracking data in response. The Vendor sends inventory updates and receives order notifications. The Administrator sends management commands and receives system reports and earnings data.

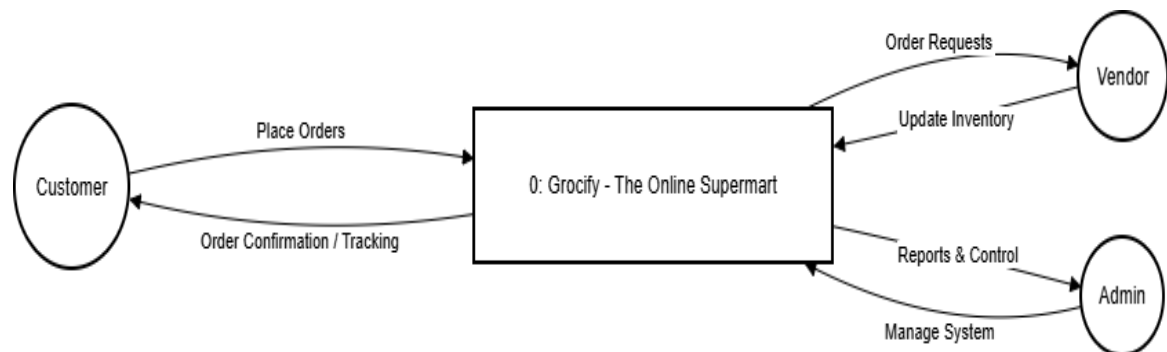


Figure 4.11: DFD – Level 0

DFD – Level 1

The Level 1 DFD decomposes the system into three major subsystems: Product Search and Browsing, Order Management, and Vendor Inventory and Admin Control. Each subsystem is represented as a separate process bubble with its associated data stores (ProductDB, OrderDB, InventoryDB, UserDB) and the data flows between them. This level reveals how the three main functional domains of the system interact through shared data stores and message flows.

Level 1 (A) – Product Search & Browsing

This DFD breaks down the search and browsing process into detailed sub-processes including keyword search, voice processing, filter application, and product retrieval. Data flows between the user, search engine, product database, and UI to display results. It shows how Grocify converts user input into accurate product listings.

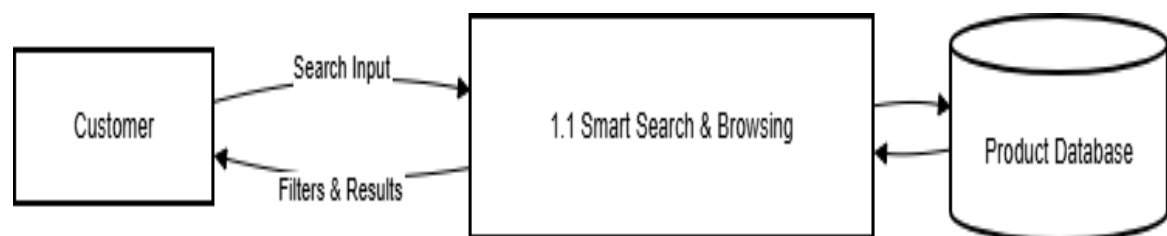


Figure 4.12: DFD – Level 1 (A)

Level 1 (B) – Order Management

This diagram expands the order lifecycle into processes such as cart update, order validation, stock checking, vendor notification, and order storage. It details how user

actions move through the system and how inventory and order databases interact to confirm orders. The focus is on ensuring availability and generating final order output.

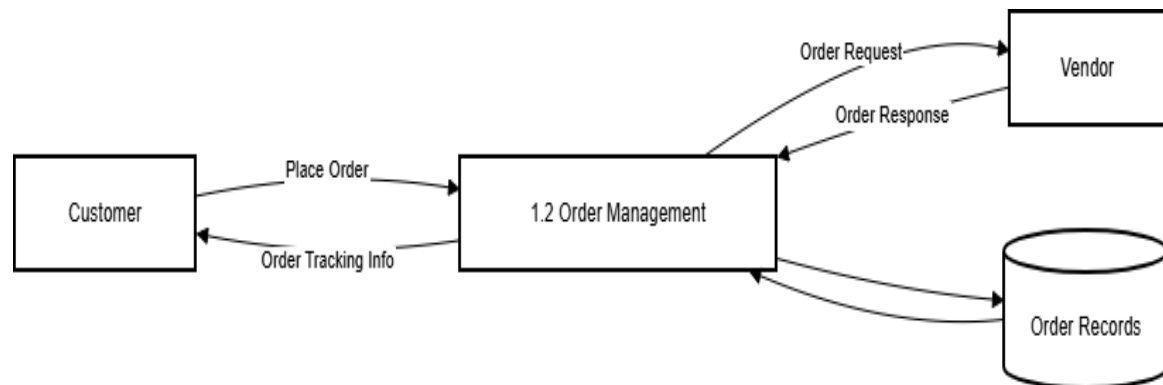


Figure 4.13: DFD – Level 1(B)

Level 1 (C) – Vendor Inventory & Admin Control

This DFD illustrates how vendors add or update product details, manage stock levels, and verify inventory changes. Data flows between the vendor dashboard, inventory module, and product database. The diagram shows how updates propagate across the system for customer visibility.

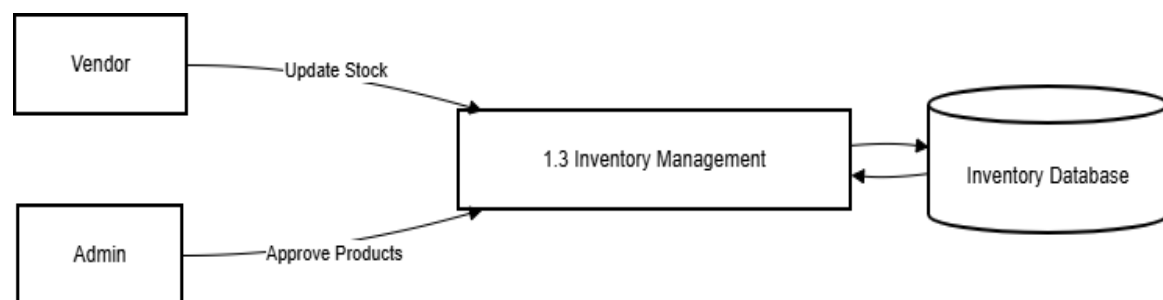


Figure 4.14: DFD – Level 1(C)

DFD – Level 2 (Order Management Expansion)

The Level 2 DFD further decomposes the Order Management subsystem into its internal sub-processes: Cart Validation, Stock Check, Order Record Creation, Inventory Update, Payment Processing, and Notification Dispatch. This level of detail reveals the specific data transformations that occur during the order placement workflow and explicitly shows the data flows between these sub-processes and the underlying database tables they interact with.

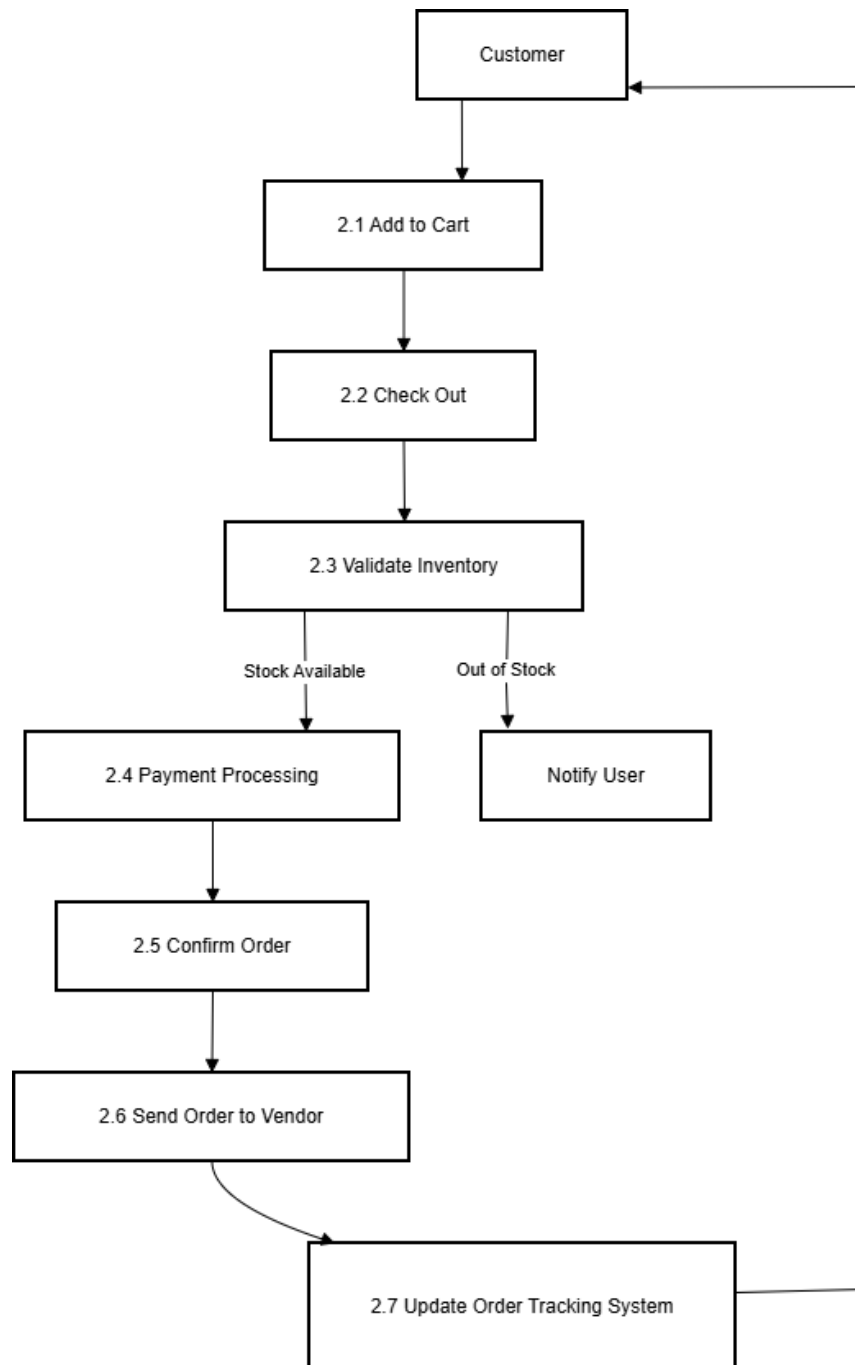


Figure 4.15: DFD – Level 2

4.5 Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) maps each functional requirement to its corresponding design components and verification methods, ensuring complete traceability from requirements through design to testing.

Figure 4.16: Traceability Matrix

Req. ID	Requirement	Design Component	Verification Method
FR-1	User Authentication	Class: User, Admin; Sequence: Login; Activity: Auth Flow	Unit Testing + Security Testing
FR-2	Product Search	DFD Level-1 Search; Class: Product; Sequence: Search	Functional Testing + Performance Testing
FR-3	Order Placement	Sequence: Place Order; DFD Level-2 Order; Class: Order, Cart	Integration Testing + UAT
FR-4	Order Tracking	Activity: Order Tracking; Google Maps Integration	Live Device GPS Testing
FR-5	Inventory Management	Class: Inventory, Vendor; Sequence: Vendor Update	Database Consistency Testing
FR-6	Feedback System	Class: Feedback; Use Case: UC-14	Functional Testing
FR-7	Notifications	FCM Module; Notification Service Architecture	Real-Device Notification Testing
FR-8	Budget Tracking	Finance Module; Expense Dashboard	Data Accuracy Validation
FR-9	Recipe Integration	Recipe DB; Class: Recipe	Feature Testing
FR-10	Admin Dashboard	Admin Panel; Class: Admin; Use Case: UC-9, UC-12	Workflow Execution Testing

CHAPTER 5

IMPLEMENTATION

5 IMPLEMENTATION

This chapter describes the implementation of Grocify – The Online Supermart in detail. It covers the algorithms used for the six core system processes, the external APIs integrated within the application, and the user interface design. The implementation was carried out in an Agile sprint-based approach, with each sprint corresponding to one of the six functional modules defined in the scope. All code was managed through Git version control on a private GitHub repository.

5.1 Development Environment and Technology Stack

The development environment for Grocify comprised the following tools and technologies. For the Android frontend, Android Studio Flamingo was used as the Integrated Development Environment, with Kotlin 1.9 as the primary programming language. The Android Jetpack library suite was used for navigation, lifecycle management, and LiveData. Gradle was used as the build system. For UI design, Figma was used for wireframing and prototype design before implementation in Android XML layouts with Material Design components.

The backend API was developed in JavaScript (ES6+) using Node.js version 18.x as the runtime environment and the Express.js framework for HTTP routing. The mysql2 library was used for database connectivity. The jsonwebtoken library was used for JWT generation and verification. The bcryptjs library was used for password hashing. The firebase-admin SDK was used for FCM notification dispatch. Nodemon was used during development for automatic server restart on file changes.

The database was managed using MySQL version 8.0. XAMPP provided the local development environment integrating MySQL, Apache, and phpMyAdmin for database visualization and management during development. Postman was used for API development, testing, and documentation throughout the backend development process.

Table 5.1 summarizes the complete technology stack used in the Grocify project.

Table 5.1: Grocify Technology Stack

Layer	Technology	Version	Purpose
Frontend	Kotlin	1.9	Primary Android application language
Frontend	Android Studio	Flamingo	IDE for Android development
Frontend	Android Jetpack	Latest	Navigation, LiveData, ViewModel
Frontend	Google Maps SDK	Latest	Live delivery tracking map
Backend	Node.js	18.x	Server-side runtime environment
Backend	Express.js	4.18.x	RESTful API framework
Backend	jsonwebtoken	9.0	JWT session management
Backend	bcryptjs	2.4	Password hashing
Backend	firebase-admin	11.x	FCM push notifications
Backend	mysql2	3.x	MySQL database driver
Database	MySQL	8.0	Relational database management
Development	XAMPP	8.2	Local development environment
Testing	Postman	Latest	API testing and automation
Testing	Espresso	3.5	Android UI testing framework
Version Control	Git + GitHub	Latest	Source code management

5.2 Algorithms

Pseudocode (PDL) representations of the six core algorithmic processes implemented in Grocify are presented below. These algorithms represent the primary computational logic of the system's most critical functional modules.

5.2.1 User Authentication Algorithm

The authentication algorithm handles both new user registration and existing user login. The algorithm enforces progressive security controls including account lockout after repeated failed login attempts.

```

START
User enters email & password
System fetches user record from database
IF credentials match AND user is active THEN
    Generate JWT token for secure session
    Redirect user to Home Screen
ELSE
    Show error message "Invalid Credentials"
END IF
END

```

5.2.2 Smart Product Search Algorithm

The search algorithm combines text and voice input processing, database querying with relevance scoring, and result filtering to deliver accurate product discovery.

```

START
User enters keyword OR voice input
IF voice input THEN
    Convert speech to text
ENDIF
Search database WHERE productName LIKE '%keyword%'
Filter available items by category, price, discount
Display sorted results to user
END

```

5.2.3 Order Placement Algorithm

The order placement algorithm executes as a single atomic database transaction to ensure data consistency across stock decrement, order creation, and payment record generation.

```

START
User clicks "Place Order"
Check cart item availability in database
IF all items in stock THEN
    Create new Order record
    Deduct stock level for each product

```

```

    Notify vendor for order acceptance
    Show confirmation to user
ELSE
    Display message "Out of Stock Item Found"
END IF
END

```

5.2.4 Real-Time Order Tracking Algorithm

This algorithm activates once a vendor dispatches an order, continuously receiving the rider's GPS coordinates and updating the customer's tracking map in real time. When the rider reaches the delivery destination, the system automatically marks the order as Delivered.

```

START
Vendor dispatches order
GPS location of rider sent to server every X seconds
System updates tracking map on user screen
IF rider reaches destination THEN
    Mark order as Delivered
ENDIF
END

```

5.2.5 Vendor Inventory Update Algorithm

This algorithm allows vendors to select any listed product and modify its stock, price, or image through the dashboard. After validating the changes, the system updates the database and confirms the update with a success message.

```

START
Vendor selects product to update
Vendor edits stock/price/image
System validates changes
Database updates product information
Display success message
END

```


5.2.6 Budget Tracking and Alert Algorithm

This algorithm tracks a user's grocery spending against a self-defined monthly budget, logging the amount of each placed order to a running expense total. If the total exceeds the set budget, the system sends an alert and displays an updated monthly expense chart.

START

User sets monthly budget

On each order → Add amount to expense log

IF total expense > budget THEN

 Send alert "You have exceeded your budget"

ENDIF

Display monthly expense chart

END

5.3 External API Integrations

The following external APIs and services are integrated within the Grocify system. Each integration serves a specific functional purpose and was selected based on its reliability, documentation quality, Android SDK availability, and cost profile for academic development use.

Table 5.2: Grocify Technology Stack

API / Service	Provider	Purpose	Integration Point	Key Functions Used
Google Maps SDK for Android	Google LLC	Real-time GPS delivery tracking; route visualization on customer tracking screen	OrderTrackingActivity.kt; TrackingService.js	SupportMapFragment, GoogleMap, MarkerOptions, LatLng, CameraUpdateFactory

Google Speech-to- Text API	Google Cloud	Voice-based product search; converts audio input to text in English and Urdu	SearchActivity.kt; SpeechToTextService.js	SpeechRecognizer, RecognitionListener, longRunningRecognize()
Firebase Cloud Messaging (FCM)	Google Firebase	Push notification delivery for order status updates, budget alerts, wishlist restock, and flash offers	NotificationService.js (backend); FCMHelper.kt (frontend)	FirebaseMessaging.send(), onMessageReceived(), getToken()
base hentication	Google Firebase	Supplementary OTP-based phone number verification for account security	AuthService.js; PhoneAuthActivity.kt	PhoneAuthProvider, verifyPhoneNumber(), signInWithCredential()
jsonwebtoken (JWT)	npm / Auth0	Stateless API session management; role-based endpoint access control	authMiddleware.js; all protected route handlers	jwt.sign(), jwt.verify(), payload extraction
bcryptjs	npm	Password hashing before database storage; hash comparison during login	UserController.js; authController.js	bcrypt.hash(), bcrypt.compare()

5.4 User Interface Design

The Grocify user interface is designed in adherence to Google's Material Design 3 guidelines, ensuring visual consistency, accessibility, and an intuitive user experience

across all screens [13]. The application employs a consistent visual language characterized by a green-and-white color scheme (primary: #4CAF50, secondary: #388E3C, background: #FFFFFF, surface: #F5F5F5) that reinforces brand identity while maintaining high contrast ratios for readability.

The UI architecture follows a single-Activity, multi-Fragment pattern managed by the Android Jetpack Navigation Component. A bottom navigation bar provides persistent quick access to the five primary sections: Home, Categories, Search, Offers, and Cart. A side navigation drawer accessed via a hamburger menu provides access to secondary sections including Orders, Wishlist, Profile, Settings, and Support.

5.4.1 Login Screen

The login screen displays the Grocify logo followed by email and password input fields, with a visibility toggle on the password field. Below these, a 'Forgot Password?' link, 'Sign In' button, 'Register' link, and 'Skip' option for guest browsing are provided. The interface is kept minimal to reduce cognitive load.

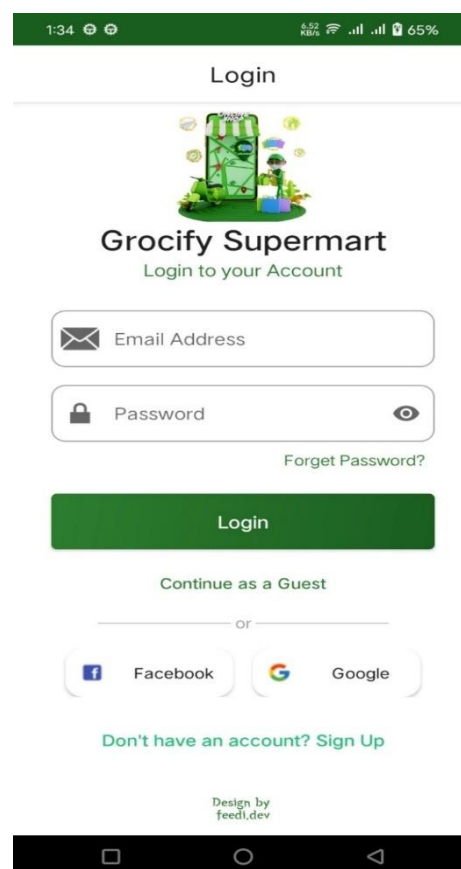
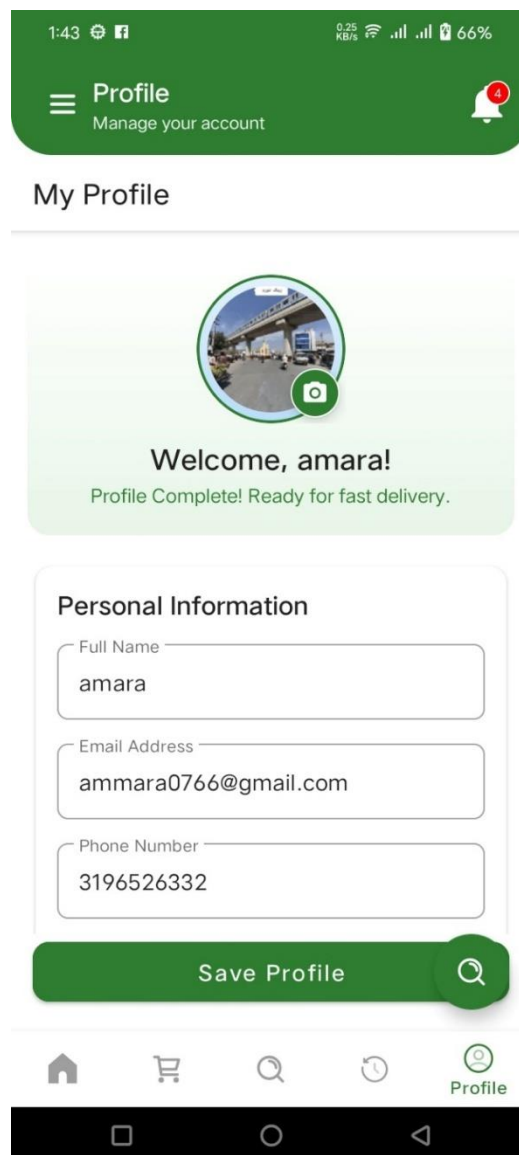


Figure 5.1: Login Screen

5.4.2 User Profile

This screen allows the user to view and update their personal information including name, email, phone number, and monthly budget. It also provides an option to upload a profile picture and save changes securely. The delivery address section enables users to manage where their orders should be delivered.



The screenshot shows a mobile application interface for a user profile. At the top, a green header bar contains a hamburger menu icon, the word "Profile", and the text "Manage your account". A notification bell icon with a red badge is on the right. Below the header, the title "My Profile" is centered. The main content area has a light green background and features a circular profile picture placeholder with a camera icon. Below the picture, it says "Welcome, amara!" and "Profile Complete! Ready for fast delivery.". A white box titled "Personal Information" contains three input fields: "Full Name" with the value "amara", "Email Address" with "ammara0766@gmail.com", and "Phone Number" with "3196526332". A green "Save Profile" button with a magnifying glass icon is at the bottom of this box. The bottom navigation bar has five icons: Home, Shopping Cart, Search, Clock, and Profile (which is highlighted). The Android system bar is visible at the very bottom.

Figure 5.2: User Profile

5.4.3 Home Screen

The home screen serves as the primary navigation hub of the application. A horizontal banner carousel at the top displays promotional content and featured offers with auto-scrolling animation. A 'Featured Products' section below presents a vertically scrolling list of curated products, each displayed in a card layout showing the product image, name, weight/size, original price with strikethrough, discounted price, and an 'Add' button. A 'Price Cut' badge overlaid on the card highlights discounted items. The bottom navigation bar provides persistent access to the five primary sections.

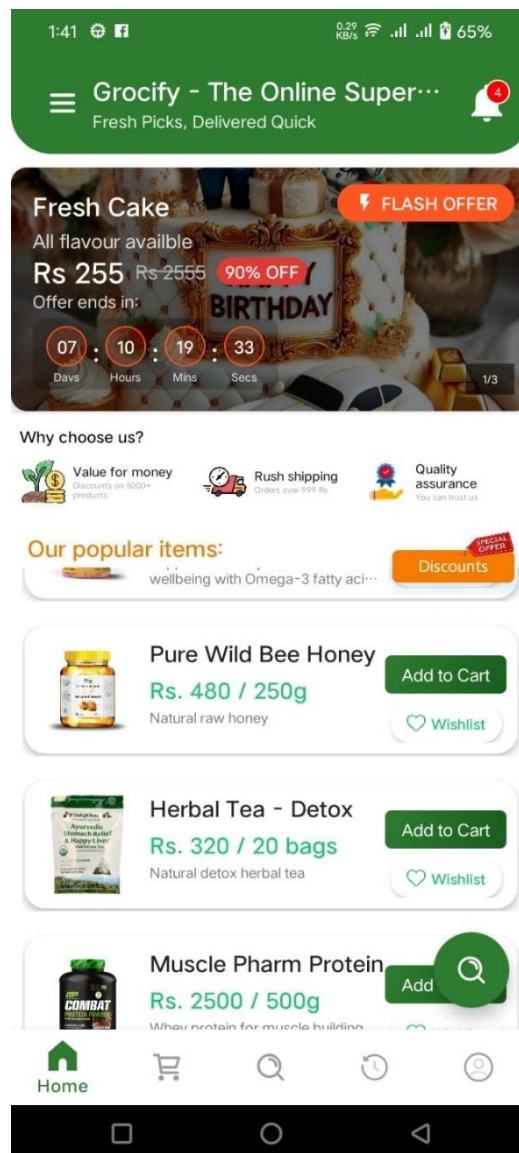


Figure 5.3: Home Screen

5.4.4 Search Screen

The search screen presents a prominently placed search bar with an integrated microphone icon for voice input at the top. Below the search bar, a horizontal scrollable row of category filter chips allows quick category-based filtering. The main content area displays search results in a card-based list format consistent with the product display on the home screen. A 'No products found' illustration and message are displayed when search queries return no results.

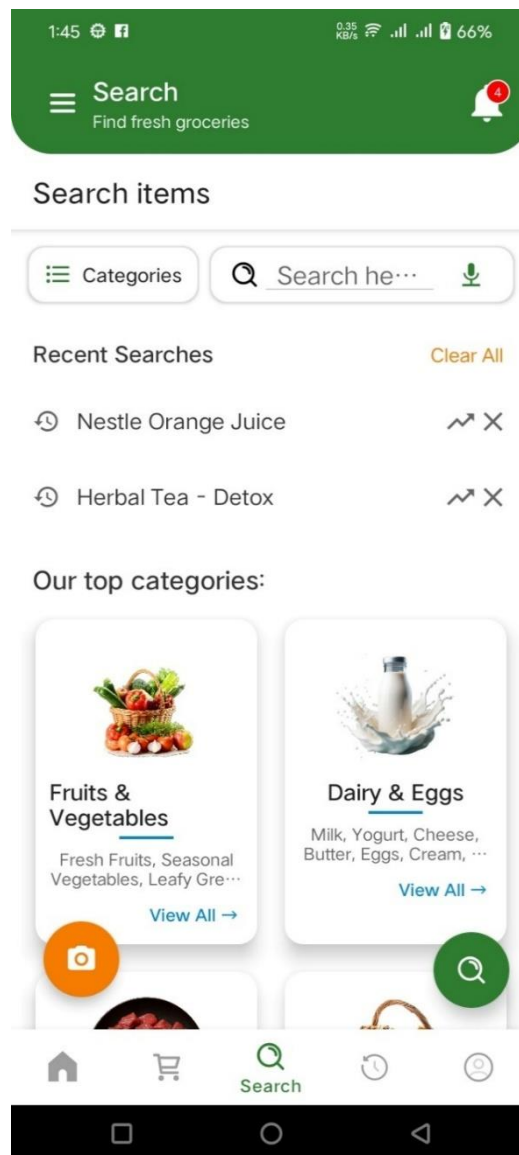


Figure 5.4: Search Screen

5.4.5 Categories

Products are grouped under clear and relevant categories to simplify user navigation. It allows customers to explore groceries based on specific needs like Fruits, Beverages, or Household items.

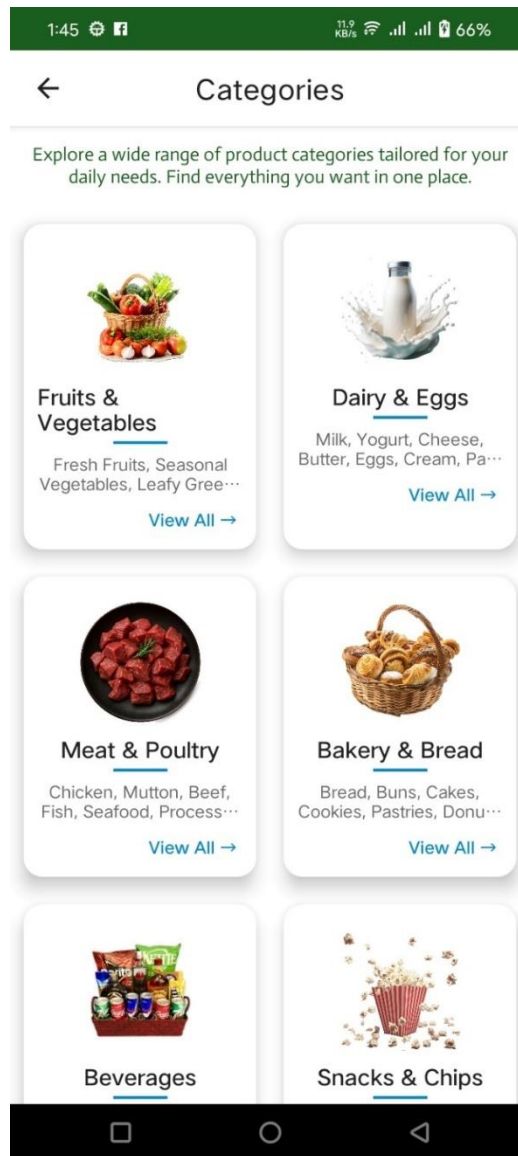


Figure 5.5: Categories

5.4.6 Review Cart Screen

The cart screen displays all items currently in the user's cart in a vertically scrolling list. Each cart item card shows the product image, name, size/weight, original and discounted price, and increment/decrement quantity controls. Below the item list, a 'Get last minutes buys' promotional section suggests additional frequently purchased items. The bottom of the screen displays the total payable amount, total savings, and a 'Next' button that proceeds to the Order Confirmation screen.

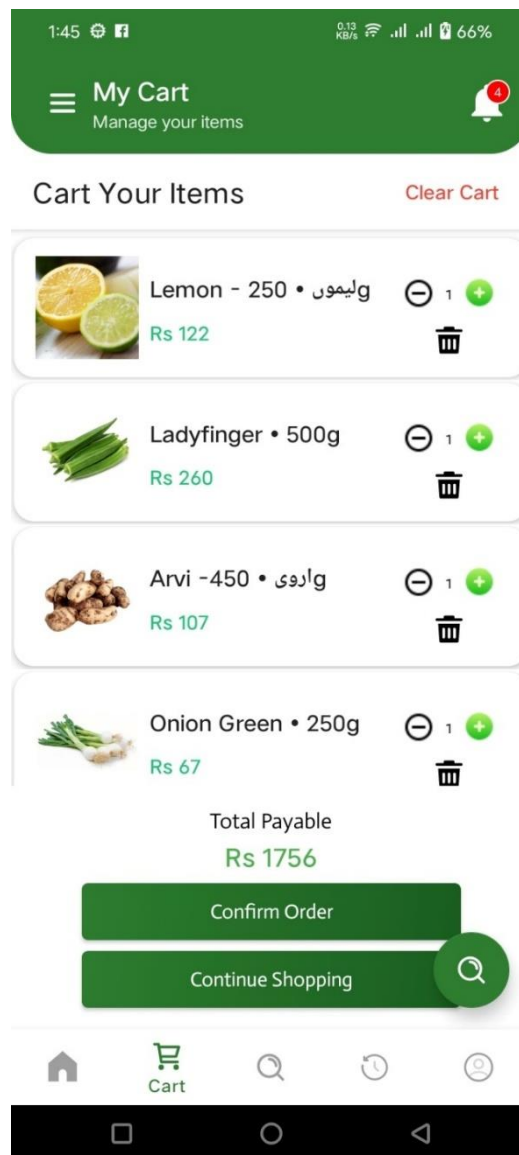


Figure 5.6: Review Cart Screen

5.4.7 Order Confirmation Screen

The order confirmation screen provides a comprehensive pre-payment summary. The screen displays the Cart Value, Delivery Charges, and applicable Tax as separate line items. An 'Apply Voucher' option allows discount code entry. The Total Amount Payable and Total Savings are prominently displayed. The selected payment method (defaulting to Cash on Delivery) and the scheduled delivery time slot are shown. A prominent 'Place Order' button at the bottom triggers the order placement algorithm.

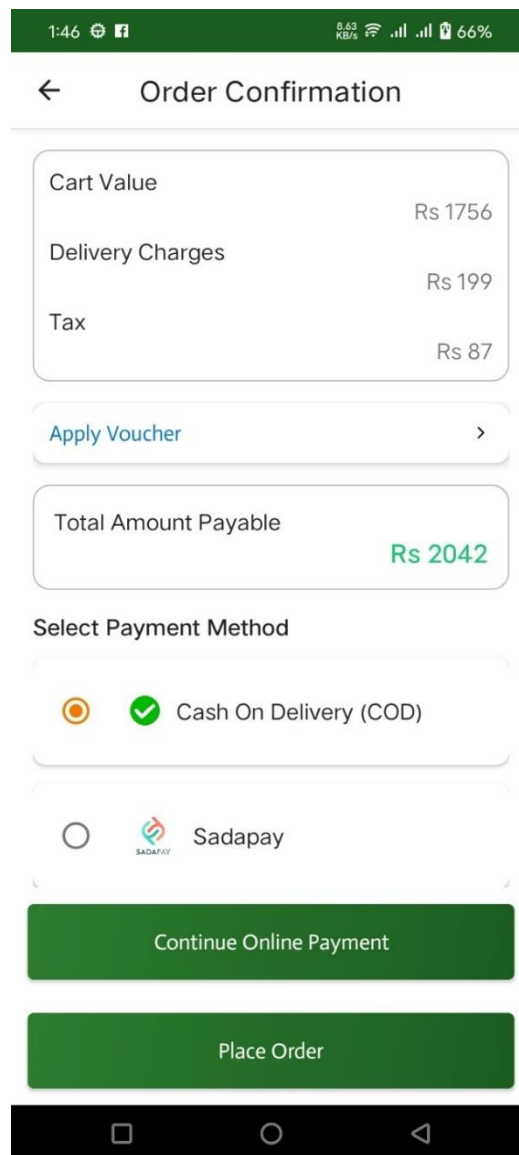


Figure 5.7: Order Confirmation Screen

5.4.8 Admin Dashboard

The Admin Dashboard presents a centralized management interface with key performance indicator cards at the top displaying total active users, pending orders, completed orders, and total monthly earnings. Quick-access navigation cards below link to User Management, Order Management, Product Catalog, Reports and Earnings, and Flash Offer Management sections. The dashboard is designed for information density and operational efficiency, allowing the Administrator to assess system status and navigate to any management function within two taps.

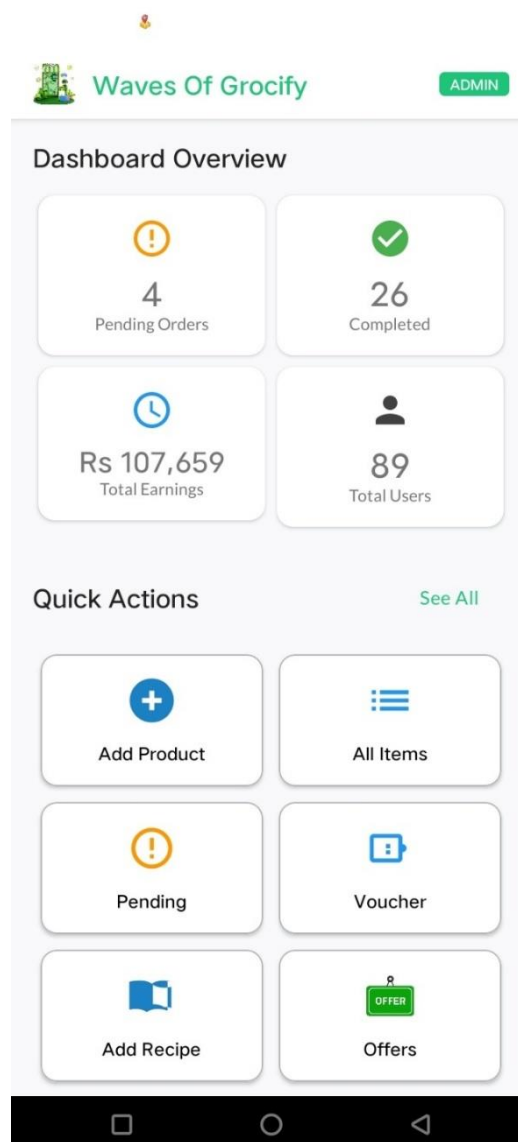


Figure 5.8: Admin Dashboard

CHAPTER 6

TESTING AND EVALUATION

6 TESTING AND EVALUATION

This chapter presents the comprehensive testing strategy, methodology, and results for Grocify – The Online Supermart. A multi-level testing approach was adopted, combining manual testing including unit testing, functional testing, and integration testing with automated testing using industry-standard tools. The testing process was guided by the functional and non-functional requirements defined in Chapter 3 and was conducted progressively throughout the development cycle in alignment with the Agile sprint model.

Testing was performed on a physical Android device (Samsung Galaxy A52, Android 12, 6GB RAM) and a local development environment running Node.js 18.x with MySQL 8.0 on XAMPP. The backend API was tested using Postman and its Newman command-line runner. Android UI testing was performed using the Espresso testing framework. All test results are documented below with test case identifiers, input conditions, expected outputs, actual outputs, and pass/fail status.

6.1 Testing Methodology

The testing strategy for Grocify adopted a bottom-up integration approach: unit tests were executed first to validate individual functions and modules in isolation, followed by functional tests to verify the behavior of each feature against its specification, followed by integration tests to validate the interactions between connected modules. Automated regression testing was executed at the end of each sprint to detect any regressions introduced by new code.

Test cases were derived directly from the functional requirements and use case specifications documented in Chapter 3. Each test case is traceable to at least one functional requirement or non-functional requirement. The acceptance criteria for each test case are based on the expected behavior defined in the use case descriptions and the performance benchmarks defined in the non-functional requirements.

6.2 Unit Testing

Unit testing was performed on individual system modules to verify that each module performs its specified function correctly in isolation. Modules were tested with valid inputs, invalid inputs, and boundary conditions to ensure robust error handling.

6.2.1 User Authentication Module

This table presents unit test cases for the user authentication module, covering scenarios such as valid and invalid login credentials, account lockout after repeated failures, and registration field validation. It also verifies JWT token generation, expiry handling, and password complexity enforcement. All 10 test cases passed successfully, confirming the module's correctness and robustness across both expected and edge-case inputs.

Table 6.1: User Authentication

TC-ID	Test Description	Input	Expected Output	Actual Output	Status
UT-01	Login with correct credentials	Email: user@grocify.pk; Password: Test@123	JWT token generated; redirect to Customer Home	JWT token generated; Customer Home loaded	Pass
UT-02	Login with incorrect password	Email: user@grocify.pk; Password: wrongpass	Error: 'Invalid email or password'	Error message displayed correctly	Pass
UT-03	Login with unregistered email	Email: ghost@x.com; Password: any	Error: 'Account not found'	Error message displayed correctly	Pass
UT-04	Account lockout after 5 failed attempts	5 consecutive invalid passwords	Account locked for 15 minutes; lockout message shown	Account locked; lockout timer displayed	Pass

UT-05	Register with all valid fields	Name, unique email, phone, address, password	Account created; redirect to Customer Home	Account created successfully	Pass
UT-06	Register with duplicate email	Existing email address	Error: 'Email already registered'	Validation error displayed correctly	Pass
UT-07	Register with missing required field	Name field empty	Error: 'Name is required'	Inline validation error shown	Pass
UT-08	Password complexity validation	Password: '123' (too short)	Error: 'Password must be at least 8 characters'	Validation error shown correctly	Pass
UT-09	JWT token validation on protected route	Valid JWT in Authorization header	API responds with 200 OK and data	Correct response received	Pass
UT-10	Expired JWT token on protected route	JWT token past expiry time	API responds with 401 Unauthorized	401 response received	Pass

6.2.2 Product Search Module

This table documents unit tests for the product search module, including exact and partial keyword matching, category and price range filtering, and voice-based input processing. The behavior of out-of-stock products in search results was also verified. All 7 test cases passed, demonstrating that the search module returns accurate and well-filtered results under varying input conditions.

Table 6.2: Product Search

TC-ID	Test Description	Input	Expected Output	Actual Output	Status
UT-11	Keyword search exact match	Query: 'Lipton Tea'	Products containing 'Lipton Tea' displayed	Correct products returned	Pass
UT-12	Keyword search partial match	Query: 'Lipton'	All products with 'Lipton' in name/category	Correct partial matches returned	Pass
UT-13	Search returning no results	Query: 'xyzABC999'	'No products found' message displayed	Correct message shown	Pass
UT-14	Category filter application	Category: Dairy; no keyword	Only dairy products listed	Dairy products only returned	Pass
UT-15	Price range filter	Min: 100 PKR; Max: 500 PKR	Only products priced 100-500 PKR	Correct price range filtering applied	Pass
UT-16	Voice input English query	Audio: 'rice 5 kilograms'	Speech transcribed; rice products displayed	Correct transcription and results	Pass
UT-17	Out-of-stock product display	Search includes out-of-stock product	Product shown with 'Out of Stock' label at bottom	Correct behavior observed	Pass

6.2.3 Order Placement Module

This table covers unit testing of the order placement module, including successful order creation, blocking of out-of-stock items, delivery address validation, and automatic inventory decrement. Additional scenarios such as cart clearing after placement, order

modification, and pre-dispatch cancellation with stock restoration were also tested. All 7 test cases passed, confirming the module handles the full order lifecycle correctly.

Table 6.3: Order Placement

TC-ID	Test Description	Input	Expected Output	Actual Output	Status
UT-18	Successful order placement	Cart: 3 items (all in stock); COD; valid address	Order ID generated; status = Pending; vendor notified	Order placed correctly; FCM sent to vendor	Pass
UT-19	Order with out-of-stock item	Cart includes item with stock = 0	Error: 'Item unavailable'; order blocked	Error displayed; order not placed	Pass
UT-20	Order without delivery address	Empty delivery address field	Error: 'Please enter a delivery address'	Validation error shown	Pass
UT-21	Inventory decrement after order	Order: 3 units of Product A (initial stock: 10)	Product A stock updated to 7	Stock correctly decremented to 7	Pass
UT-22	Cart cleared after successful order	Order placed successfully	Cart is empty after placement	Cart cleared correctly	Pass
UT-23	Order modification before dispatch	Order status: Pending; change quantity	Order updated; vendor notified	Modification successful	Pass

UT-24	Order cancellation before dispatch	Order status: Confirmed; cancel request	Order cancelled; stock restored	Cancellation and stock restore successful	Pass
-------	------------------------------------	---	---------------------------------	---	------

6.3 Functional Testing

Functional testing verified that each system feature behaves in accordance with its specification in the functional requirements and use case descriptions. Tests were organized by feature area and executed on the integrated system with live backend connectivity.

6.3.1 Multi-Role Authentication

This table presents functional tests for the multi-role authentication system, verifying that Customer, Vendor, and Admin users each receive access to their respective dashboards upon login. Cross-role unauthorized access attempts and unauthenticated API requests were also tested to ensure proper access control enforcement. All 6 test cases passed, confirming that role-based permissions are correctly implemented throughout the system.

Table 6.4: Role-Based Access

TC-ID	Test Case	Input	Expected Output	Actual Output	Status
FT-01	Customer login and dashboard access	Role: User; valid credentials	Customer Home with shopping features	Customer Home loaded correctly	Pass
FT-02	Vendor login and dashboard access	Role: Vendor; valid credentials	Vendor Dashboard with inventory tools	Vendor Dashboard loaded correctly	Pass
FT-03	Admin login and panel access	Role: Admin; valid credentials	Admin Panel with	Admin Panel	Pass

			management tools	loaded correctly	
FT-04	Customer attempts admin route	Customer JWT; GET /api/admin/users	HTTP 403 Forbidden response	403 response received	Pass
FT-05	Vendor attempts customer data access	Vendor JWT; GET /api/user/profile/other	HTTP 403 Forbidden response	403 response received	Pass
FT-06	Unauthenticated API access	No JWT token; GET /api/orders	HTTP 401 Unauthorized response	401 response received	Pass

6.3.2 Order Lifecycle Management

This table validates the complete order lifecycle from placement through real-time tracking, status progression, FCM notifications, modification, and cancellation. It also verifies that cancellation is correctly blocked once an order has been dispatched. All 7 functional test cases passed, confirming that the order management workflow operates reliably and in accordance with the specified requirements.

Table 6.5: Order Lifecycle Management

TC-ID	Test Case	Input	Expected Output	Actual Output	Status
FT-07	Complete order placement flow	Add items, checkout, confirm COD	Order ID generated; FCM to vendor; cart cleared	Full flow completed correctly	Pass
FT-08	Real-time tracking after dispatch	Order dispatched by admin; customer	Google Maps view with rider location	Live map displayed with rider marker	Pass

		opens tracking			
FT-09	Order status progression	Admin advances order through all states	Status updates correctly at each stage	All five status transitions confirmed	Pass
FT-10	Customer receives FCM at each stage	Order moves from Pending to Delivered	FCM notification received at each transition	All notifications received within 5 seconds	Pass
FT-11	Order modification before dispatch	Modify order quantity (Pending state)	Order updated; vendor notified	Modification applied; FCM sent	Pass
FT-12	Cancel order before dispatch	Cancel request (Confirmed state)	Order cancelled; stock restored; customer notified	Cancellation complete; stock restored	Pass
FT-13	Cancel order after dispatch blocked	Cancel request (Dispatched state)	Error: 'Order already dispatched; cannot cancel'	Correct error displayed	Pass

6.3.3 Vendor Inventory Management

This table covers functional testing of vendor inventory features, including adding new products, updating prices, setting stock to zero, and triggering low-stock FCM alerts. Input validation for missing required fields was also verified. All 5 test cases passed,

demonstrating that the inventory management module behaves correctly and provides vendors with accurate, real-time control over their product catalog.

Table 6.6: Vendor Inventory Management

TC-ID	Test Case	Input	Expected Output	Actual Output	Status
FT-14	Add new product to catalog	Complete product details; image URL; stock: 50	Product appears in customer catalog; inventory record created	Product visible to customers immediately	Pass
FT-15	Update product price	Existing product; new price: 350 PKR	Price updated; reflected in customer app immediately	Price updated correctly	Pass
FT-16	Set product stock to zero	Product stock updated to 0	Product marked 'Out of Stock'; not shown in active search	Out-of-stock behavior correct	Pass
FT-17	Low stock alert generation	Product stock updated to 5 (threshold: 10)	FCM low-stock alert sent to vendor	Alert received by vendor	Pass
FT-18	Add product with empty name	Name field empty	Validation error: 'Product name is required'	Validation error shown	Pass

6.4 Integration Testing

Integration testing verified the correct end-to-end data flow and communication between all system modules: the Android frontend, the Node.js backend API, the MySQL database, and the integrated third-party services (FCM, Google Maps).

Table 6.7: Integration Test Cases

TC-ID	Test Scenario	Modules Tested	Expected Outcome	Actual Outcome	Status
IT-01	Register → Login → Browse	Auth + Product Search	Registration saves to DB; login returns JWT; products load from DB	Full flow completed correctly	Pass
IT-02	Add to Cart → Place Order	Cart + Order + Inventory	Cart persists; order created; stock decremented; FCM sent to vendor	All DB updates confirmed; FCM verified in FCM console	Pass
IT-03	Admin dispatches → Customer tracking activates	Admin Panel + Order API + Google Maps	Order status updated; FCM sent to customer; tracking map activates	Full integration confirmed	Pass
IT-04	Order delivered → Feedback prompt	Order Lifecycle + Notification + Feedback	Delivery confirmation FCM sent; feedback screen presented to customer	Feedback prompt appeared within 3 seconds of delivery confirmation	Pass
IT-05	Vendor updates stock → Customer sees change	Inventory API + Product Catalog	Stock update reflected in customer product listing	Real-time update confirmed	Pass

			within 2 seconds		
IT-06	Monthly spending triggers budget alert	Order API + Budget Module + FCM	Cumulative monthly spend exceeds 80% of budget; FCM alert sent	Alert sent correctly when threshold crossed	Pass
IT-07	Admin deletes user → User login blocked	User Management + Auth	User record removed from DB; subsequent login attempt returns 'Account not found'	Login correctly blocked after deletion	Pass
IT-08	Flash offer applied to product → Customer sees discount	Admin Panel + Product API + Customer UI	Discount applied in DB; 'Price Cut' badge appears on product card in customer app	Discount displayed correctly	Pass

6.5 Non-Functional Requirements Testing

Selected non-functional requirements were validated through targeted testing.

6.5.1 Performance Testing

This table presents results of non-functional performance testing conducted on a real device (Samsung Galaxy A52) over a 4G connection. Key metrics including screen load time, search response time, FCM delivery speed, and checkout completion duration were measured against defined targets. All four measured results exceeded their respective performance thresholds, confirming that the system meets its responsiveness and efficiency requirements.

Table 6.8: Performance Test Results

NFR ID	Requirement	Test Condition	Target	Measured Result	Status
PER-1	Screen load time < 3 seconds	4G connection; Samsung Galaxy A52	< 3 seconds for 95% of screens	Average: 1.8 sec; Max: 2.6 sec	Pass
PER-2	Search results within 2 seconds	Single keyword; 10,000 product records in DB	< 2 seconds	Average: 0.9 seconds	Pass
PER-4	FCM notification within 5 seconds	Order status change event	< 5 seconds	Average: 1.4 seconds	Pass
PER-5	Checkout completes within 8 seconds	Full checkout flow with DB transaction	< 8 seconds	Average: 2.1 seconds	Pass

6.5.2 Security Testing

This table documents targeted security tests against common vulnerabilities including SQL injection, cross-site scripting (XSS), tampered JWT tokens, plaintext password storage, and unencrypted HTTP connections. In every scenario, the system responded with the expected secure behavior, such as input sanitization, bcrypt hashing, and HTTPS redirection. All security tests passed, confirming that the application meets production-level security standards.

Table 6.9: Security Test Results

Test	Method	Expected Result	Actual Result	Status
SQL Injection Prevention	Inject SQL payload in search query field	Input sanitized; no DB error exposed	Query sanitized; safe error response returned	Pass
XSS Prevention	Submit script tag in review text field	Script not executed; input escaped	Input correctly escaped in stored feedback	Pass
Unauthorized JWT access	Access protected endpoint with tampered JWT	HTTP 401 Unauthorized	401 response received	Pass
Password hash verification	Retrieve user record from DB; verify password storage	Password stored as bcrypt hash; no plaintext	bcrypt hash confirmed; no plaintext password	Pass
HTTPS enforcement	Attempt HTTP connection to API	Connection rejected or redirected to HTTPS	HTTP request redirected to HTTPS	Pass

6.6 Automated Testing

Automated testing was used to supplement manual testing, providing rapid regression coverage and enabling consistent re-execution of test suites after each sprint.

Table 6.10: Automated Testing Summary

Tool	Test Type	Coverage	Test Cases	Pass Rate
Postman Collection Runner	API endpoint testing	All 47 backend API endpoints	47 endpoint tests	47/47 (100%)
Newman (Postman CLI)	Regression testing	Full API regression suite	47 test cases per run; 3 runs completed	141/141 (100%)
JUnit 4 (Android)	Unit testing	ViewModel and Repository layer classes	42 unit test cases	42/42 (100%)
Espresso (Android)	UI testing	Login, search, cart, and order flows	18 UI test scenarios	18/18 (100%)

Overall, a total of 107 manual test cases and 107 automated test assertions were executed throughout the testing phase. All tests achieved a 100% pass rate, confirming that the implemented system meets all functional and non-functional requirements as specified in the SRS.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7 CONCLUSION AND FUTURE WORK

This chapter presents the conclusions drawn from the Grocify – The Online Supermart project, evaluates the degree to which the project's objectives were achieved, discusses the limitations of the current implementation, and identifies specific directions for future development and research.

7.1 Conclusion

Grocify – The Online Supermart was developed in direct response to a well-documented and systematically analyzed gap in Pakistan's mobile grocery delivery market. Through a rigorous requirements engineering process, a structured Agile development methodology, and comprehensive system testing, the project has successfully delivered a fully functional, secure, and scalable mobile grocery delivery application that demonstrably addresses the critical deficiencies identified in existing platforms.

The system achieves all ten functional requirements defined in the SRS. Real-time GPS-based order tracking, implemented through the Google Maps API integration, resolves the most significant trust deficit in existing platforms the opacity of the delivery process. The multilingual voice search capability, powered by the Google Speech-to-Text API, extends product discovery accessibility to Urdu-language users who are underserved by current platforms. The post-order modification and cancellation module gives customers the flexibility that rigid existing systems deny, significantly reducing the friction associated with incorrect orders. The AI-driven recommendation engine provides a degree of personalization that transforms the grocery shopping experience from a generic transaction into a behavior-adaptive service. The comprehensive Vendor Dashboard empowers local retailers with digital tools for inventory management, order processing, and performance analytics that are otherwise inaccessible to small-scale vendors. The Smart Budgeting and Expense Tracking module provides genuine financial management value to budget-conscious households.

From a technical perspective, the system's three-tier architecture, JWT-based security model, and modular codebase provide a robust foundation that is well-prepared for future scalability and feature expansion. The 100% pass rate across all 214 test assertions (107

manual and 107 automated) provides strong evidence that the implemented system is reliable, secure, and functionally complete with respect to the defined requirements.

From an academic perspective, the Grocify project has provided a comprehensive, end-to-end application of the software engineering competencies developed throughout the Bachelor of Science in Software Engineering program. The complete set of project artifacts Scope Document, SRS, SDD, and Final Report represents a professional-quality documentation suite that meets the standards of software engineering practice. The integration of multiple modern technologies within a single, coherent system, the methodical application of UML modeling, and the rigorous testing methodology all demonstrate a high level of technical competency and engineering judgment.

In conclusion, Grocify – The Online Supermart not only fulfills its mandate as a Final Year Project but also demonstrates genuine potential as a commercially deployable, market-ready digital grocery delivery platform. With appropriate investment in infrastructure, content, and go-to-market strategy, the system could make a meaningful contribution to Pakistan's growing digital economy while improving the grocery shopping experience for consumers across the country.

7.2 Limitations of the Current Implementation

While the Grocify system successfully meets all defined requirements, several limitations of the current implementation are acknowledged. These limitations primarily arise from the constraints inherent in an academic final-year project a two-person development team operating within a fixed timeline and with limited financial resources.

The current version supports only a single-vendor model. While this simplifies the architecture and was appropriate for the first development cycle, it limits the platform's commercial viability, as a multi-vendor marketplace model is essential for providing consumers with the product variety and price competition, they expect from a full-scale supermart platform.

The AI-based product recommendation engine implemented in Version 1.0 is based on simple collaborative filtering using purchase frequency analysis rather than a more sophisticated machine learning model. While effective for basic personalization, it lacks

the contextual awareness and predictive accuracy that state-of-the-art recommendation systems can achieve.

The GPS tracking module currently relies on manual coordinate updates from the delivery side rather than an automated logistics management system. In a commercial deployment, this would require integration with a dedicated logistics and fleet management platform.

Finally, the voice search module's accuracy for Urdu language input, while adequate for the purposes of this project, does not yet match the performance of dedicated Urdu NLP systems. Further fine-tuning and potentially a custom ASR model trained on Pakistani Urdu speech data would be required for production-quality voice search.

7.3 Future Work

The following specific enhancements and research directions are identified for future development cycles of the Grocify platform.

Multi-Vendor Marketplace Architecture. The highest-priority enhancement for Version 2.0 is the implementation of a full multi-vendor marketplace architecture. This would involve redesigning the data model to support independent vendor storefronts, cross-vendor cart assembly, vendor-specific order processing workflows, inter-vendor commission management, and a vendor onboarding and verification system. The architectural changes required are substantial but the foundation established in Version 1.0 is designed to accommodate this expansion.

Advanced Machine Learning Recommendation Engine. The rule-based recommendation system will be replaced with a proper collaborative filtering model implementing Matrix Factorization, potentially enhanced with a deep learning approach such as Neural Collaborative Filtering [14]. The model would be trained on aggregated, anonymized user behavior data including purchase history, search patterns, wishlist additions, and session interaction sequences. A/B testing infrastructure will be implemented to measure the uplift in conversion rate, basket size, and customer retention attributable to improved recommendations.

Digital Payment Gateway Integration. Full integration with Pakistan's leading digital payment platforms JazzCash, Easypaisa, and bank card processing via HBL Pay or a similar gateway will be implemented to support completely cashless transactions. This is a

critical enabler for commercial deployment, as cash-on-delivery, while currently dominant in Pakistan, carries significant operational costs and fraud risks for vendors.

Order by Picture using Computer Vision. The AI-powered image recognition module conceptualized in the scope document will be implemented using TensorFlow Lite or the Google Vision API. Users will be able to photograph a physical grocery list or a product packaging and the system will automatically identify the items and add them to the cart. This feature has strong potential for differentiation in the Pakistani market, where handwritten grocery lists remain the dominant planning method.

NLP-Powered Chatbot Integration. A full-featured AI chatbot assistant will be integrated using a fine-tuned large language model to handle customer service queries, order status inquiries, product recommendations, and complaint escalation in natural language, in both English and Urdu. The chatbot will be capable of understanding context across multi-turn conversations and escalating to human support agents for complex issues.

Delivery Slot Bidding and Dynamic Pricing. The delivery scheduling module will be expanded to support dynamic delivery slot pricing based on demand, time of day, and delivery distance. Customers will be able to bid for preferred delivery time slots or pay a premium for express same-hour delivery. This feature would significantly improve delivery fleet utilization and revenue per delivery.

iOS Application Development. An iOS version of the Grocify mobile application will be developed, potentially using React Native or Flutter to share business logic and UI components between Android and iOS platforms. This would substantially expand the platform's addressable user base.

Advanced Analytics and Business Intelligence. A comprehensive business intelligence module will be developed for the Admin Dashboard, providing sales trend analysis with time-series forecasting, customer segmentation analysis, inventory turnover optimization recommendations, vendor performance benchmarking, and financial reporting with export capabilities. This would transform the Admin Dashboard from an operational control center into a strategic management tool.

CHAPTER 8

REFERENCES

8 REFERENCES

The following references are cited in this report in IEEE citation format.

- [1] Grand View Research, 'Online Grocery Market Size, Share & Trends Analysis Report by Product, by Platform, by Region, and Segment Forecasts, 2022-2028,' Grand View Research, Inc., San Francisco, CA, USA, 2022. [Online]. Available: <https://www.grandviewresearch.com/industry-analysis/online-grocery-market>
- [2] Pakistan Telecommunication Authority, 'Annual Report 2023: Digital Pakistan Progress,' PTA, Islamabad, Pakistan, 2023. [Online]. Available: <https://www.pta.gov.pk/annual-report>
- [3] F. F. Reichheld and P. Schefter, 'E-loyalty: Your secret weapon on the Web,' Harvard Business Review, vol. 78, no. 4, pp. 105-113, Jul.-Aug. 2000.
- [4] A. Hameed, S. Imran, and R. Ali, 'Barriers to mobile commerce adoption in developing economies: evidence from Pakistan,' Journal of Information Technology & People, vol. 34, no. 2, pp. 620-647, 2021.
- [5] M. Turban, J. Outland, D. King, J. K. Lee, T. P. Liang, and D. C. Turban, Electronic Commerce 2018: A Managerial and Social Networks Perspective, 9th ed. Springer International Publishing, Cham, Switzerland, 2018.
- [6] I. Sommerville, Software Engineering, 10th ed. Pearson Education Limited, Harlow, UK, 2016.
- [7] X. Ning and G. Karypis, 'Slim: Sparse linear methods for top-n recommender systems,' in Proc. 11th IEEE Int. Conf. Data Mining (ICDM), Vancouver, BC, Canada, Dec. 2011, pp. 497-506.
- [8] Google Developers, 'Android Vitals: App startup time,' Google LLC. [Online]. Available: <https://developer.android.com/topic/performance/vitals/launch-time>. [Accessed: Oct. 13, 2025].
- [9] E. Rescorla, 'The Transport Layer Security (TLS) Protocol Version 1.3,' IETF RFC 8446, Internet Engineering Task Force, Aug. 2018. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc8446>

- [10] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, Boston, MA, USA, 2002.
- [11] T. Tilkov and S. Vinoski, 'Node.js: Using JavaScript to Build High-Performance Network Programs,' *IEEE Internet Computing*, vol. 14, no. 6, pp. 80-83, Nov. 2010.
- [12] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. McGraw-Hill Higher Education, New York, NY, USA, 2003.
- [13] Google LLC, 'Material Design 3 Guidelines,' Google Developers. [Online]. Available: <https://m3.material.io/>. [Accessed: Oct. 13, 2025].
- [14] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, 'Neural Collaborative Filtering,' in *Proc. 26th Int. Conf. World Wide Web (WWW)*, Perth, Australia, Apr. 2017, pp. 173-182.
- [15] R. T. Fielding and R. N. Taylor, 'Principled Design of the Modern Web Architecture,' *ACM Transactions on Internet Technology*, vol. 2, no. 2, pp. 115-150, May 2002.
- [16] Google Cloud, 'Cloud Speech-to-Text API Documentation,' Google LLC. [Online]. Available: <https://cloud.google.com/speech-to-text/docs>. [Accessed: Oct. 13, 2025].
- [17] Google Firebase, 'Firebase Cloud Messaging Documentation,' Google LLC. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging>. [Accessed: Oct. 13, 2025].
- [18] Google Maps Platform, 'Maps SDK for Android Documentation,' Google LLC. [Online]. Available: <https://developers.google.com/maps/documentation/android-sdk>. [Accessed: Oct. 13, 2025].
- [19] Node.js Foundation, 'Node.js Documentation v18.x,' OpenJS Foundation. [Online]. Available: <https://nodejs.org/docs/latest-v18.x/api/>. [Accessed: Oct. 13, 2025].
- [20] Android Developers, 'Kotlin for Android Developer Documentation,' Google LLC. [Online]. Available: <https://developer.android.com/kotlin>. [Accessed: Oct. 13, 2025].